IntelQcopter: Mathematical Model, Six Control Loops, State Estimation and Navigation
Detailed derivations and reproducible simulation evidence

IntelQcopter project

20 October 2026

Contents

1 Scope, evidence and physical assumptions **1**
- 1.1 What this document establishes . . . 1
- 1.2 Evidence boundaries . . . 2

2 Stochastic modelling foundations **3**
- 2.1 Covariance and its transformation . . . 3
- 2.2 Height kinematics from direct time integration . . . 3
- 2.3 Two distinct process-noise conventions . . . 3

3 Coordinate frames and rigid-body mechanics **5**
- 3.1 Frames, rotations and signs . . . 5
- 3.2 Newton and Euler equations . . . 6
- 3.3 Motor geometry, allocation and saturation . . . 6
- 3.4 Numerical propagation . . . 7

4 Constructing the flight model step by step **8**
- 4.1 Start from a free-body diagram, not a guessed transfer function . . . 8
- 4.2 Write the nonlinear differential equations before linearizing . . . 8
- 4.3 Find the hover equilibrium by substitution . . . 9
- 4.4 Linearize by explicitly retaining first-order terms . . . 9
- 4.5 Build each open-loop transfer function from the differential equation . . . 10
- 4.6 Relate normalized thrust to a possible rotor-speed model . . . 10

5 Closed-loop transfer functions and stability calculations **11**
- 5.1 Rate PI: derive the denominator without skipping the feedback algebra . . . 11
- 5.2 Roll/pitch angle cascade: retain the inner loop dynamics . . . 11
- 5.3 Altitude PID and the distinction between height and velocity requests . . . 11
- 5.4 What second-order parameters actually mean . . . 12
- 5.5 Routh–Hurwitz: construct and evaluate the stability conditions . . . 12
- 5.6 A continuous parameter-interval proof for the conservative candidate . . . 13
- 5.7 The horizontal position loop is sixth order . . . 14
- 5.8 Discrete-time stability at the real control interval . . . 14
- 5.9 Include the 20 Hz companion with the 500 Hz flight loop . . . 15
- 5.10 Frequency-domain margins and uncertainty . . . 15

6 Companion missions, boost and actuator authority **17**
- 6.1 Keep one flight-control mode; move mission sequencing upstream . . . 17
- 6.2 What arming and RT boost mean . . . 17
- 6.3 Prove the motor-headroom allocation interval . . . 18

7 Exactly six feedback controllers **19**
7.1 Signals and units . . . 19
7.2 Roll and pitch rate PI derivation . . . 19
7.3 Outer angle loops and yaw . . . 20
7.4 One altitude PID with vertical-speed stick command . . . 20
7.5 A reproducible gain family, not hidden hand tuning . . . 21

8 Kalman filtering: proof and implemented equations **22**
8.1 Derivation of the gain . . . 22
8.2 Scalar one-angle filter . . . 22
8.3 Actual MPU filter: one axis, two states . . . 23
8.4 Accelerometer observations and their limitation . . . 23
8.5 Altitude filter: state, measurements and expanded products . . . 24
8.6 Joseph multiplication in full detail . . . 24
8.7 Gravity compensation, lidar and observability . . . 25

9 Uncertainty, statistical checks and error accounting **26**
9.1 What standard deviation does and does not mean . . . 26
9.2 Innovation consistency and physical-parameter errors . . . 26
9.3 Separate error sources . . . 27

10 Companion navigation and perception **28**
10.1 A bounded static-map baseline . . . 28
10.2 ROS 2 data contract . . . 28
10.3 Why YOLO is not a range sensor . . . 29

11 Reproduction and next physical measurements **30**

12 Recorded numerical results **31**
12.1 Six PID gain tables . . . 31
12.1.1 Baseline . . . 31
12.1.2 Selected simulation . . . 31
12.2 Selection and route suite . . . 31
12.3 Kalman gains, standard deviation and distributions . . . 34
12.3.1 Independent Monte Carlo experiments . . . 34
12.4 Numerical refinement and algebra checks . . . 36
12.5 Executed ROS 2 communication test . . . 36
12.6 Executed YOLO and USD tests . . . 36
12.7 Files and reproducibility . . . 37

13 New ground-to-ground garden experiments and numerical analysis **38**
13.1 What changed from the original report . . . 38
13.2 Current six-controller candidate and model coefficients . . . 38
13.3 Poles, step response, Routh values and sampling . . . 39
13.4 The uncertainty grid exposes a real tuning limitation . . . 40
13.5 Camera-only panel distance, derived from the measurement geometry . . . 40
13.6 Training and operating-domain limitations . . . 41
13.7 Recorded mission progression and outcomes . . . 42
13.8 ROS 2 selection, timing and implementation boundaries . . . 44
13.9 Current missing information and unresolved problems . . . 44

Chapter 1

Scope, evidence and physical assumptions

1.1 What this document establishes

This report derives an explicit six-degree-of-freedom model, a six-controller architecture, the covariance and gain calculations used by the C estimators, and a bounded local-navigation baseline. Numerical results are generated by the production C control equations and a common C++ navigation core. They establish behaviour under stated simulation assumptions. They do not identify the actual aircraft or certify a flight tune.

The physical aircraft is an F450/Q450-style X quadcopter with 9047 propellers, nominal 1400 KV motors, four 40 A ESCs and a 3S 4200 mAh LiPo. The approximately 150 g battery mass is unverified. A motor's KV value and an ESC's current rating do not determine thrust, propeller efficiency or inertia. In particular, 40 A is not a thrust measurement. The following complete-vehicle model is an explicit test assumption, not the previous frame-plus-battery mass subtotal.

Quantity	Test value	Unit
Complete mass m	1.2	kg
Diagonal motor spacing L	0.45	m
$J = \mathrm{diag}(J_x, J_y, J_z)$	$(0.012, 0.012, 0.022)$	kg m^2
Maximum thrust per motor $T_{\max}$	6	N
Reaction torque coefficient κ	0.015	m
Thrust response time τ_m	0.04	s
Linear drag coefficient d_v	0.12	N s/m
Gravity g	9.80665	m/s^2
Control/physics interval Δ	0.002	s
Barometer update rate	50	Hz
Gyro sample standard deviation	0.2	deg/s
Accelerometer sample standard deviation	0.005	g per axis
Barometric height standard deviation	0.05	m

The collision envelope uses $L/2 + 0.2286/2 = 0.3393$ m, rounded to 0.34 m. This includes the 9-inch propeller radius. Using $L/2$ alone would overstate obstacle clearance by approximately 11.5 cm. The actual geometry and propeller clearance must be measured; the circle is conservative in plan view.

1.2 Evidence boundaries

1. The CPU plant integrates rigid-body motion, motor lag and sensor noise. The shared library calls the same C functions as the flight firmware's math core. Host compilation is not RP2350 instruction emulation or hardware timing.

2. Two Python ROS 2 adapter nodes exchange test envelopes through Fast DDS UDP and invoke that C library. The C++ planner is compiled as a shared library. The ROS-specific C++ wrapper requires a full ROS development environment and has not been compiled here.

3. YOLO11n inference is tested independently on an official reference image. The original airborne baseline route uses a known static obstacle map and simulated local odometry, not YOLO-generated obstacle ranges.

4. Isaac Sim launch attempts failed before a healthy simulation server was available: physical-device enumeration/GPU initialization failed. There is no successful Isaac physics run or Isaac recording in these results. The delivered video is an original visualization of logged CPU simulation data.

5. Physical motors remain disabled. No UART, LoRa, physical IMU, ESC, or real camera flight pipeline is exercised by the tests in this report.

Chapter 2

Stochastic modelling foundations

2.1 Covariance and its transformation

Let x have mean μ and covariance $P = \mathbb{E}[(x-\mu)(x-\mu)^\mathsf{T}]$. For $y = Ax + b$,

$$y - \mathbb{E}[y] = A(x-\mu), \tag{2.1}$$

$$\mathrm{Cov}(y) = \mathbb{E}[A(x-\mu)(x-\mu)^\mathsf{T}A^\mathsf{T}] = APA^\mathsf{T}. \tag{2.2}$$

For independent zero-mean process noise w of covariance Q, $\mathrm{Cov}(Ax+w) = APA^\mathsf{T} + Q$: the two cross expectations vanish by independence and zero means. If the noise is correlated with the state, this last step is invalid and cross-covariance terms must be retained.

Symmetry follows because $P_{ij} = \mathbb{E}[e_i e_j] = P_{ji}$. For every v, $v^\mathsf{T}Pv = \mathbb{E}[(v^\mathsf{T}e)^2] \geq 0$, so a covariance is positive semidefinite. For a symmetric 2×2 matrix, necessary and sufficient conditions are $P_{00} \geq 0$, $P_{11} \geq 0$ and $P_{00}P_{11} - P_{01}^2 \geq 0$. Covariance units vary by entry; for height and speed, P_{00} is m^2, P_{01} is m^2/s and P_{11} is $\mathrm{m}^2/\mathrm{s}^2$.

2.2 Height kinematics from direct time integration

If acceleration a_k is constant during one interval, integrate $\dot{v} = a_k$ from 0 to t to obtain $v(t) = v_k + a_k t$. Integrate $\dot{h} = v(t)$ from zero to Δ: $h_{k+1} = h_k + v_k\Delta + a_k\Delta^2/2$, while $v_{k+1} = v_k + a_k\Delta$. Thus the altitude state update uses

$$F = \begin{bmatrix} 1 & \Delta \\ 0 & 1 \end{bmatrix}, \qquad G = \begin{bmatrix} \Delta^2/2 \\ \Delta \end{bmatrix}. \tag{2.3}$$

These are the particular physical kinematic equations; no general matrix multiplication or matrix-exponential tutorial is included.

2.3 Two distinct process-noise conventions

For one independent acceleration error η_k held constant for an entire sample, $\mathrm{Var}(\eta_k) = \sigma_a^2$ and $w_k = G\eta_k$. Thus

$$Q_{\text{held}} = \sigma_a^2 GG^\mathsf{T} = \sigma_a^2 \begin{bmatrix} \Delta^4/4 & \Delta^3/2 \\ \Delta^3/2 & \Delta^2 \end{bmatrix}. \tag{2.4}$$

For continuous white acceleration with spectral density q_c, however,

$$Q_{\text{continuous}} = \int_0^\Delta e^{A\tau} B q_c B^\mathsf{T} e^{A^\mathsf{T}\tau} \mathrm{d}\tau \tag{2.5}$$

$$= q_c \int_0^\Delta \begin{bmatrix} \tau^2 & \tau \\ \tau & 1 \end{bmatrix} \mathrm{d}\tau = q_c \begin{bmatrix} \Delta^3/3 & \Delta^2/2 \\ \Delta^2/2 & \Delta \end{bmatrix}. \tag{2.6}$$

These are different stochastic models with different units. The altitude C implementation uses (2.4), with $\sigma_a = 1.5$ m/s^2 per 2 ms sample. This is not a continuous spectral density. Changing the sample rate while holding that number fixed changes the effective uncertainty per second. Real noise parameters require measured bandwidth and timing.

Chapter 3

Coordinate frames and rigid-body mechanics

3.1 Frames, rotations and signs

World W is a local right-handed frame with z upward. Body B uses forward, left, up (FLU). $R = R_z(\psi)R_y(\theta)R_x(\phi)$ maps body vectors to world vectors. Here ϕ, θ, ψ are roll, pitch, yaw in radians for mechanics. The elementary matrices are

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\phi & -s_\phi \\ 0 & s_\phi & c_\phi \end{bmatrix}, \quad R_y = \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix}, \tag{3.1}$$

$$R_z = \begin{bmatrix} c_\psi & -s_\psi & 0 \\ s_\psi & c_\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}, \tag{3.2}$$

where $c_\alpha = \cos\alpha$ and $s_\alpha = \sin\alpha$. Multiplication yields

$$R = \begin{bmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix}. \tag{3.3}$$

For example, entry $(3,3)$ is $c_\theta c_\phi$, so upward thrust contributes $T c_\theta c_\phi$ to vertical force. At zero yaw and small angles, positive pitch accelerates forward and positive roll accelerates right, which is negative world y when body and world headings align. This explains the signs in the companion acceleration-to-attitude conversion.

Body angular velocity is $\omega = [p, q, r]^\mathsf{T}$. It is not generally the Euler-angle derivative:

$$\begin{bmatrix} \dot\phi \\ \dot\theta \\ \dot\psi \end{bmatrix} = \begin{bmatrix} 1 & s_\phi \tan\theta & c_\phi \tan\theta \\ 0 & c_\phi & -s_\phi \\ 0 & s_\phi/c_\theta & c_\phi/c_\theta \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix}. \tag{3.4}$$

Near hover $p \simeq \dot\phi$, $q \simeq \dot\theta$, $r \simeq \dot\psi$. The independent roll/pitch filters and gain design use that approximation; the plant itself propagates a unit quaternion. Aggressive attitude changes require a coupled quaternion estimator and appropriate rate-setpoint mapping.

3.2 Newton and Euler equations

Let p_W be position, v_W velocity, and $e_3 = [0, 0, 1]^\mathsf{T}$. Then

$$\dot{p}_W = v_W, \tag{3.5}$$

$$m\dot{v}_W = Re_3 \sum_{i=1}^{4} T_i - mge_3 - d_v v_W + f_{\text{wind}}, \tag{3.6}$$

$$J\dot{\omega} = \tau - \omega \times (J\omega). \tag{3.7}$$

The rotational equation follows from angular-momentum balance. If $h_B = J\omega$, then the inertial derivative expressed in rotating body axes is $\dot{h}_B + \omega \times h_B = \tau_B$. For diagonal inertia,

$$J_x\dot{p} = \tau_x + (J_y - J_z)qr, \tag{3.8}$$

$$J_y\dot{q} = \tau_y + (J_z - J_x)rp, \tag{3.9}$$

$$J_z\dot{r} = \tau_z + (J_x - J_y)pq. \tag{3.10}$$

The inertia tensor must be positive definite and satisfy physical principal moment inequalities; the provisional diagonal values do. A measured model requires component masses and positions, using

$$J = \sum_i \left(R_i J_{i,\text{COM}} R_i^\mathsf{T} + m_i[(r_i^\mathsf{T} r_i)I - r_i r_i^\mathsf{T}] \right). \tag{3.11}$$

This is the parallel-axis theorem. It separates each component's own inertia from the extra moment caused by its distance from the vehicle centre of mass.

3.3 Motor geometry, allocation and saturation

Set $a = L/(2\sqrt{2}) = 0.159099$ m. Logical motor order is front-left (FL), front-right (FR), rear-right (RR), rear-left (RL), at $(a, a, 0), (a, -a, 0), (-a, -a, 0), (-a, a, 0)$ respectively. For $r_i = (x_i, y_i, 0)$ and thrust $f_i = (0, 0, T_i)$,

$$r_i \times f_i = \begin{bmatrix} y_i T_i \\ -x_i T_i \\ 0 \end{bmatrix}. \tag{3.12}$$

Assigning alternating reaction-torque signs $(-, +, -, +)$ gives

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ a & -a & -a & a \\ -a & -a & a & a \\ -\kappa & \kappa & -\kappa & \kappa \end{bmatrix} \begin{bmatrix} T_{FL} \\ T_{FR} \\ T_{RR} \\ T_{RL} \end{bmatrix}. \tag{3.13}$$

These are assumed spin directions; wiring must eventually be checked. In the software, u_i is a normalized *desired thrust proxy*, not a verified PWM, DShot, RPM or electrical-power relationship. Define

$$\begin{bmatrix} u_{FL} \\ u_{FR} \\ u_{RR} \\ u_{RL} \end{bmatrix} = \begin{bmatrix} 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} c \\ u_\phi \\ u_\theta \\ u_\psi \end{bmatrix}. \tag{3.14}$$

Before saturation and lag, $T_i = T_{\max} u_i$, hence

$$T = 4T_{\max} c, \quad \tau_x = 4aT_{\max} u_\phi, \quad \tau_y = 4aT_{\max} u_\theta, \quad \tau_z = 4\kappa T_{\max} u_\psi. \tag{3.15}$$

For example, positive pitch must increase rear thrust and decrease front thrust. A host regression test now checks this sign. The original allocator clipped outputs independently; the current implementation scales infeasible differential spans and shifts collective into a feasible interval, as derived in the boost/authority chapter. Saturation still limits physical authority.

Motor lag is represented by $\tau_m \dot{u}_i^{\text{actual}} + u_i^{\text{actual}} = u_i$. For constant commanded u_i over Δ, solve the scalar differential equation using an integrating factor:

$$u_{k+1}^{\text{actual}} = e^{-\Delta/\tau_m} u_k^{\text{actual}} + (1 - e^{-\Delta/\tau_m}) u_k. \tag{3.16}$$

This exponential update is exact for the assumed first-order actuator. Actual motors may need $T = k_T \Omega^2$, $\tau = k_Q \Omega^2$ and a measured electrical model. No battery sag or propeller aerodynamics are currently identified.

3.4 Numerical propagation

Position uses $p_{k+1} = p_k + \Delta v_k + \frac{1}{2}\Delta^2 a_k$ and velocity uses $v_{k+1} = v_k + \Delta a_k$. Angular velocity uses a first-order update of the Euler equation. Given the updated ω held over a step,

$$\delta q = \begin{bmatrix} \cos(\alpha/2) \\ \hat{\omega} \sin(\alpha/2) \end{bmatrix}, \quad \alpha = \|\omega\|\Delta, \quad q_{k+1} = q_k \otimes \delta q, \tag{3.17}$$

with the zero-speed limit handled continuously and the quaternion normalized. For $q = (s, v)$ and $r = (t, w)$, multiplication is $q \otimes r = (st - v^{\mathsf{T}} w,\ sw + tv + v \times w)$. This preserves rotational geometry for the held angular velocity, but the entire plant integrator is not an exact solution of coupled nonlinear motion. Timestep refinement and empirical identification are separate requirements.

Chapter 4

Constructing the flight model step by step

4.1 Start from a free-body diagram, not a guessed transfer function

Our first modelling choice is a system boundary: the complete flying aircraft, including battery, frame, motors, electronics and propellers. Its total mass is $m = \sum_i m_i$, and its centre of mass is $r_C = \sum_i m_i r_i / m$. Choosing the body origin at r_C removes the extra moment caused by expressing angular momentum about an accelerating offset point. Neither a battery mass nor a frame mass is the complete m in Newton's law.

Choose a local world frame with upward unit vector e_3 and a forward-left-up body frame. Let the four thrust forces be $T_i e_3$ in body coordinates. There are three categories of external force: rotated rotor thrust, weight, and aerodynamic/disturbance force. Internal frame forces cancel when summing over the system boundary. Thus the force balance is constructed as

$$\sum f_W = \underbrace{Re_3(T_1 + T_2 + T_3 + T_4)}_{\text{rotors}} - \underbrace{mge_3}_{\text{weight}} - \underbrace{d_v v_W}_{\text{linear drag}} + f_w, \tag{4.1}$$

$$\dot{p}_W = v_W, \qquad m\dot{v}_W = \sum f_W. \tag{4.2}$$

Dividing by m produces acceleration, not velocity. Integrating acceleration once gives velocity and a second time gives position. This is why a translational position transfer function has two low-frequency integrations before damping, control or other dynamics are introduced.

For each motor at $r_i = (x_i, y_i, 0)$, the torque about the centre of mass is $(y_i T_i, -x_i T_i, 0)$. Add rotor drag-reaction torques with alternating signs. The sum is the allocation equation derived in the mechanics chapter. Newton's angular counterpart is $J\dot{\omega} + \omega \times J\omega = \tau$. We keep the cross term in the nonlinear plant and explicitly neglect its second-order perturbation when constructing the hover linear model.

4.2 Write the nonlinear differential equations before linearizing

Write $T = \sum T_i$ and use the third column of R. The translational equations are, component by component,

$$m\ddot{x} = T(c_\psi s_\theta c_\phi + s_\psi s_\phi) - d_v \dot{x} + f_{wx}, \tag{4.3}$$

$$m\ddot{y} = T(s_\psi s_\theta c_\phi - c_\psi s_\phi) - d_v \dot{y} + f_{wy}, \tag{4.4}$$

$$m\ddot{h} = T c_\theta c_\phi - mg - d_v \dot{h} + f_{wz}. \tag{4.5}$$

For diagonal inertia,

$$\dot{p} = [\tau_x + (J_y - J_z)qr]/J_x, \tag{4.6}$$
$$\dot{q} = [\tau_y + (J_z - J_x)rp]/J_y, \tag{4.7}$$
$$\dot{r} = [\tau_z + (J_x - J_y)pq]/J_z. \tag{4.8}$$

These six acceleration equations need kinematics as well: $\dot{p}_W = v_W$ and quaternion attitude propagation. The state is position, velocity, attitude and angular velocity; four additional motor-lag states make the actuator dynamics explicit. Quaternion norm is constrained to one, so four quaternion coordinates represent three independent orientation degrees of freedom.

4.3 Find the hover equilibrium by substitution

An equilibrium has zero time derivatives. Choose $v_0 = 0$, $\omega_0 = 0$, $\phi_0 = \theta_0 = 0$ and any fixed heading ψ_0. Substitution in the vertical equation gives $0 = T_0 - mg$, so $T_0 = mg$. Zero torque with a symmetric allocation gives $T_{i0} = mg/4$. If normalized desired thrust satisfies $T_i = T_{\max} u_i$, then

$$u_{i0} = c_0 = \frac{mg}{4T_{\max}}. \tag{4.9}$$

For $m = 1.2$ kg and $T_{\max} = 6$ N, each motor must supply 2.941995 N and $c_0 = 0.4903325$. This is a model calculation, not an identified ESC duty cycle. The thrust-to-weight ratio of this assumed model is $24/11.76798 \simeq 2.0394$. It is not evidence that the real 1400KV/9047 combination has that reserve.

4.4 Linearize by explicitly retaining first-order terms

Define perturbations $\delta T = T - T_0$, $\delta\phi = \phi - \phi_0$ and so on. The first-order Taylor expansion of $\sin\epsilon$ is ϵ; that of $\cos\epsilon$ is 1. Products such as $\delta T\delta\theta$ contain two small perturbations and are second order. At heading zero,

$$(T_0 + \delta T)\sin\delta\theta\cos\delta\phi \simeq T_0\delta\theta, \tag{4.10}$$
$$-(T_0 + \delta T)\sin\delta\phi \simeq -T_0\delta\phi, \tag{4.11}$$
$$(T_0 + \delta T)\cos\delta\theta\cos\delta\phi - mg \simeq \delta T. \tag{4.12}$$

Using $T_0/m = g$, define $\gamma = d_v/m = 0.1\ \mathrm{s}^{-1}$ and obtain

$$\delta\ddot{x} + \gamma\,\delta\dot{x} = g\,\delta\theta + f_{wx}/m, \tag{4.13}$$
$$\delta\ddot{y} + \gamma\,\delta\dot{y} = -g\,\delta\phi + f_{wy}/m, \tag{4.14}$$
$$\delta\ddot{h} + \gamma\,\delta\dot{h} = \delta T/m + f_{wz}/m. \tag{4.15}$$

Here γ is the damping coefficient and δx denotes a perturbation. Below we omit the perturbation prefix to keep the equations legible. At a nonzero heading, rotate the two horizontal acceleration equations by the heading angle. This is a local linearization, not a claim that Euler coordinates work globally.

The body-rate products qr, rp, pq vanish to first order about zero rates. Near level attitude $\dot{\phi} \simeq p$, $\dot{\theta} \simeq q$, $\dot{\psi} \simeq r$. Thus $J_x\ddot{\phi} = \tau_x$, $J_y\ddot{\theta} = \tau_y$ and $J_z\dot{r} = \tau_z$. There is intentionally no yaw-angle feedback requirement.

4.5 Build each open-loop transfer function from the differential equation

A transfer function describes a linear input/output relation with zero initial perturbations. For $\dot{y} = f$, the unilateral Laplace transform is $sY(s) - y(0) = F(s)$. For $\ddot{y}$, it is $s^2Y(s) - sy(0) - \dot{y}(0)$. Setting the initial perturbations to zero removes those initial-condition terms; it does not mean the actual aircraft has no initial state. Initial-condition responses must be added separately.

From $J_x\ddot{\phi} = \tau_x$, first transform to $J_x s^2\Phi = \mathcal{T}_x$, then divide both sides by $J_x s^2\mathcal{T}_x$:

$$\frac{\Phi(s)}{\mathcal{T}_x(s)} = \frac{1}{J_x s^2}, \quad \frac{P(s)}{\mathcal{T}_x(s)} = \frac{1}{J_x s}. \tag{4.16}$$

Use the allocation $\tau_x = 4aT_{\max}u_\phi$ and the motor lag $\tau_m\dot{u}_{\text{act}} + u_{\text{act}} = u_{\text{cmd}}$. Transforming the latter gives $(\tau_m s + 1)U_{\text{act}} = U_{\text{cmd}}$. Therefore the cascade is

$$G_p(s) = \frac{P_d(s)}{U_\phi(s)} = \frac{b_x}{s(\tau_m s + 1)}, \qquad G_\phi(s) = \frac{\Phi_d(s)}{U_\phi(s)} = \frac{b_x}{s^2(\tau_m s + 1)}, \tag{4.17}$$

where $b_x = (4aT_{\max}/J_x)(180/\pi)$ because the controller uses degrees. Pitch has b_y instead; yaw rate has $b_\psi = 4\kappa T_{\max}(180/\pi)/J_z$. The rate plant is second order with lag and the angle plant third order. Without actuator lag these reduce to first and second order respectively.

For altitude, $\ddot{h} + \gamma\dot{h} = b_h u_{\text{act}}$ with $b_h = 4T_{\max}/m = 20$ m/s^2/command. Transforming each derivative gives

$$G_h(s) = \frac{H(s)}{U_h(s)} = \frac{b_h}{s(s + \gamma)(\tau_m s + 1)}. \tag{4.18}$$

This retains drag and actuator lag. The often-used double integrator b_h/s^2 is obtained only after explicitly neglecting both. Horizontal position from small angle is $X/\Theta = g/[s(s+\gamma)]$ and $Y/\Phi = -g/[s(s+\gamma)]$ in radians. Degree-valued angle inputs require an additional factor $\pi/180$.

4.6 Relate normalized thrust to a possible rotor-speed model

A better identified actuator could use $T = k_T\Omega^2$ and $Q = k_Q\Omega^2$. Around nonzero hover speed Ω_0, expand

$$k_T(\Omega_0 + \Delta\Omega)^2 = k_T\Omega_0^2 + 2k_T\Omega_0\Delta\Omega + k_T(\Delta\Omega)^2. \tag{4.19}$$

Discarding the quadratic perturbation gives $\Delta T = 2k_T\Omega_0\Delta\Omega$. A first-order identified command-to-speed gain $k_\Omega/(\tau_\Omega s + 1)$ then yields $\Delta T/\Delta u = 2k_T\Omega_0 k_\Omega/(\tau_\Omega s + 1)$. The current normalized-thrust model replaces that unknown product with $T_{\max}$. KV alone supplies none of $k_T, k_Q, k_\Omega, \tau_\Omega$ under load. A higher-order model is useful only when its extra parameters are measured.

Chapter 5

Closed-loop transfer functions and stability calculations

5.1 Rate PI: derive the denominator without skipping the feedback algebra

Let $G = b/[s(\tau_m s+1)]$, $C = K_p + K_i/s$ and $e = r_c - r$. Then $r = GCe = GC(r_c - r)$. Move the feedback term to the left: $(1+GC)r = GCr_c$. Divide by $1+GC$ and clear denominators:

$$T_r(s) = \frac{r}{r_c} = \frac{b(K_p s + K_i)}{\tau_m s^3 + s^2 + bK_p s + bK_i}. \tag{5.1}$$

The numerator zero at $s = -K_i/K_p$ changes the step response; denominator poles alone do not completely describe overshoot. With $\tau_m = 0$, matching $s^2 + 2\zeta\omega_n s + \omega_n^2$ gives $K_p = 2\zeta\omega_n/b$ and $K_i = \omega_n^2/b$. This is an initial design followed by checking the cubic, not an exact second-order description of the complete plant.

5.2 Roll/pitch angle cascade: retain the inner loop dynamics

Let $k_a = 3\ \mathrm{s}^{-1}$ and $p_c = k_a(\phi_c - \phi)$. Since $p = s\phi$,

$$\phi = \frac{T_r}{s} k_a(\phi_c - \phi), \tag{5.2}$$

$$\left(1 + \frac{k_a T_r}{s}\right)\phi = \frac{k_a T_r}{s}\phi_c, \tag{5.3}$$

$$T_\phi(s) = \frac{k_a T_r}{s + k_a T_r}. \tag{5.4}$$

Substitute the rate-loop numerator and denominator and collect powers:

$$\boxed{T_\phi(s) = \frac{bk_a(K_p s + K_i)}{\tau_m s^4 + s^3 + bK_p s^2 + b(K_i + k_a K_p)s + bk_a K_i}.} \tag{5.5}$$

Pitch is identical only under the assumed $J_x = J_y$. This fourth-order expression is more complete than replacing the inner rate loop by unity. The ideal outer approximation $k_a/(s+k_a)$ is justified only well below the inner closed-loop bandwidth, away from saturation and estimator error.

5.3 Altitude PID and the distinction between height and velocity requests

At fixed equilibrium, $e_h = h_c - h$, $\dot{I}_h = e_h$ and $u = K_p e_h + K_i I_h + K_d(v_c - v)$, where the internally integrated target satisfies $\dot{h}_c = v_c$. For differentiable consistent references, $V_c = sH_c$, $V = sH$

and $I_h = (H_c - H)/s$, so

$$U = (K_p + K_i/s + K_d s)(H_c - H). \tag{5.6}$$

Combine this with G_h above and multiply the characteristic equation by $s^2(s+\gamma)(\tau_m s + 1)$:

$$D_h(s) = s^2(s+\gamma)(\tau_m s + 1) + b_h(K_d s^2 + K_p s + K_i) \tag{5.7}$$

$$= \tau_m s^4 + (1 + \gamma\tau_m)s^3 + (\gamma + b_h K_d)s^2 + b_h K_p s + b_h K_i, \tag{5.8}$$

$$\boxed{T_h(s)} = \boxed{\frac{b_h(K_d s^2 + K_p s + K_i)}{D_h(s)}}. \tag{5.9}$$

The transfer from commanded vertical velocity to height is $T_h(s)/s$; the transfer to vertical velocity is $T_h(s)$, with zero initial perturbations. A discontinuous height step accompanied by an impulsive derivative reference is not the actual stick interface. The mission generates bounded v_c; the height target integrates it. If one instead steps h_c while forcing $v_c = 0$, the numerator is $b_h(K_p s + K_i)$: the denominator is the same, but the reference response differs. This prevents applying a textbook PID step response to the wrong command interface.

When lag is neglected but drag retained, matching $(s + p_h)(s^2 + 2\zeta_h\omega_h s + \omega_h^2)$ gives

$$K_d = (2\zeta_h\omega_h + p_h - \gamma)/b_h, \tag{5.10}$$

$$K_p = (\omega_h^2 + 2p_h\zeta_h\omega_h)/b_h, \tag{5.11}$$

$$K_i = p_h\omega_h^2/b_h. \tag{5.12}$$

The existing baseline was designed with $\gamma = 0$, giving $K_d = 0.175$. Including the assumed drag gives $K_d = 0.170$. The selected $\lambda = 1.3$ candidate similarly gives 0.2275 versus 0.2225. These differences are shown as model alternatives; gains are not silently changed without a new route test.

5.4 What second-order parameters actually mean

For a complex pair $s_{1,2} = -\sigma \pm j\omega_d$ with $\sigma > 0$, $\omega_n = \sqrt{\sigma^2 + \omega_d^2}$ and $\zeta = \sigma/\omega_n$. For the ideal normalized second-order low-pass without zeros,

$$M_p = e^{-\pi\zeta/\sqrt{1-\zeta^2}}, \quad t_p = \pi/\omega_d, \quad t_{s,2\%} \simeq 4/(\zeta\omega_n). \tag{5.13}$$

These follow from the decaying sinusoidal homogeneous response and its extrema; the settling approximation bounds its exponential envelope approximately. For example the ideal rate design $\zeta = 0.9$, $\omega_n = 12$ has $\omega_d = 5.2307$ rad/s and envelope settling about 0.3704 s. Its ideal no-zero overshoot is about 0.152%. The actual PI closed loop has a zero, and actuator lag adds another pole. Therefore these figures describe the chosen target polynomial, not a guaranteed measured aircraft step response. The companion numerical analysis computes actual step responses for the complete local transfer functions, including their zeros.

5.5 Routh–Hurwitz: construct and evaluate the stability conditions

For $a_3 s^3 + a_2 s^2 + a_1 s + a_0$ with $a_3 > 0$, the Routh first column is

$$a_3, \quad a_2, \quad \frac{a_2 a_1 - a_3 a_0}{a_2}, \quad a_0. \tag{5.14}$$

All entries must be strictly positive for strict left-half-plane roots, assuming no degenerate zero rows. Applying this to the rate loop gives positive gains and $K_p > \tau_m K_i$. Substituting the

pole-matched PI gives $\omega_n < 2\zeta/\tau_m$. For $\tau_m = 0.04$ s the bound is 45 rad/s; the selected rate frequency 15.6 rad/s meets this local cubic condition.

For $a_4 s^4 + a_3 s^3 + a_2 s^2 + a_1 s + a_0$, construct

$$\begin{array}{c|ccc} s^4 & a_4 & a_2 & a_0 \\ s^3 & a_3 & a_1 & 0 \\ s^2 & \Delta_2/a_3 & a_0 & 0 \\ s^1 & \Delta_3/\Delta_2 & 0 & 0 \\ s^0 & a_0 & 0 & 0 \end{array}, \tag{5.15}$$

where $\Delta_2 = a_3 a_2 - a_4 a_1$ and $\Delta_3 = a_1 \Delta_2 - a_3^2 a_0$. With $a_4, a_3, a_0 > 0$, strict stability requires $\Delta_2 > 0$ and $\Delta_3 > 0$. Positive polynomial coefficients alone are insufficient. Substitution for the full angle cascade gives

$$\Delta_2 = bK_p - \tau_m b(K_i + k_a K_p), \tag{5.16}$$

$$\Delta_3 = b(K_i + k_a K_p)\Delta_2 - bk_a K_i. \tag{5.17}$$

For altitude use $a_3 = 1 + \gamma\tau_m$, $a_2 = \gamma + b_h K_d$, $a_1 = b_h K_p$ and $a_0 = b_h K_i$. The executable analysis reports each determinant, every pole and the actual step metrics for both gain sets. These strict tests apply to the local unsaturated models. Ground contact, mode transitions and a changing camera map are not a single LTI system.

5.6 A continuous parameter-interval proof for the conservative candidate

The grid finds counterexamples. For $\lambda = 0.7$ we can additionally prove local Routh inequalities over a continuous box. Assume $b \in [0.65b_0, 1.35b_0]$, $\tau \in [0.02, 0.09]$ s, and for height $\gamma \in [0.08, 0.13]$ s^{-1}. These are assumed ranges, not measured confidence intervals for the real aircraft.

For the angle cascade define $u = K_i + k_a K_p > 0$ and $A(\tau) = K_p - \tau u$. Since $u > 0$, A decreases with τ, so $A \geq A_{\min} = K_p - \tau_{\max} u$. The determinants become

$$\Delta_2 = bA, \qquad \Delta_3 = b[buA - k_a K_i]. \tag{5.18}$$

If $A_{\min} > 0$, multiplication by positive b, u preserves inequalities. It is therefore sufficient to check

$$A_{\min} > 0, \qquad b_{\min} u A_{\min} - k_a K_i > 0. \tag{5.19}$$

The numerical lower bounds are 0.000257095 and 0.00776085 approximately. All coefficients and Routh first-column entries are consequently positive throughout that continuous box for this decoupled angle model. Yaw requires $K_p - \tau_{\max} K_i > 0$, with lower bound 0.00462303.

For height, expand the determinant without skipping terms:

$$\Delta_2 = (1 + \gamma\tau)(\gamma + bK_d) - \tau bK_p \tag{5.20}$$

$$= \gamma + \gamma^2\tau + bK_d + b\gamma\tau K_d - b\tau K_p. \tag{5.21}$$

Since the discarded $\gamma^2\tau$ and $b\gamma\tau K_d$ terms are nonnegative, and $K_d - \tau_{\max} K_p > 0$ for this candidate,

$$\Delta_2 \geq \gamma_{\min} + b_{\min}(K_d - \tau_{\max} K_p) = D_{2,\mathrm{lb}} > 0. \tag{5.22}$$

The next determinant obeys

$$\Delta_3 = b\left[K_p \Delta_2 - K_i(1 + \gamma\tau)^2\right] \tag{5.23}$$

$$\geq b\left[K_p D_{2,\mathrm{lb}} - K_i(1 + \gamma_{\max}\tau_{\max})^2\right]. \tag{5.24}$$

Here $D_{2,\mathrm{lb}} \simeq 1.5460874$ and the final bracket is approximately $0.1354504 > 0$. This proves strict local height-loop stability over the specified continuous box. It excludes saturation, nonlinear coupling, estimator dynamics, contacts, delays and companion guidance. Those exclusions are part of the theorem.

The fast candidate also has an analytic counterexample. For $\lambda = 1.3$, $K_i/K_p = 8.6667\ \mathrm{s}^{-1}$. The necessary angle condition $A > 0$ implies $k_a < 1/\tau - K_i/K_p$. At $\tau = 0.09$ s the right side is $2.4444\ \mathrm{s}^{-1}$, below the chosen $k_a = 3$. Thus this Routh condition fails regardless of positive plant gain at that lag. Nominal success at 0.04 s does not establish stability at 0.09 s.

5.7 The horizontal position loop is sixth order

The companion generates $a_c = k_x(x_c - x) - k_v\dot{x}$ and $\theta_c = a_c/g$ in radians. Use $X/\Theta = g/[s(s+\gamma)]$ and the complete angle cascade $T_\theta = N_\theta/D_\theta$. Substitution gives

$$X = \frac{T_\theta}{s(s+\gamma)}[k_x X_c - (k_x + k_v s)X], \tag{5.25}$$

$$[s(s+\gamma)D_\theta + (k_v s + k_x)N_\theta]X = k_x N_\theta X_c, \tag{5.26}$$

$$\boxed{\frac{X}{X_c}} = \boxed{\frac{k_x N_\theta}{s(s+\gamma)D_\theta + (k_v s + k_x)N_\theta}}. \tag{5.27}$$

This denominator is sixth order. Navigation uses $k_x = 1.8\ \mathrm{s}^{-2}$, $k_v = 1.8\ \mathrm{s}^{-1}$; approach/landing uses $k_x = 1.2$ and the same k_v. The lateral-roll sign cancels with its guidance-conversion sign, giving the same local scalar polynomial under the symmetric model. This is upstream guidance; there are still exactly six flight-controller PID objects.

For $D_x = d_6 s^6 + d_5 s^5 + \cdots + d_0$, the specific Hurwitz array is

$$\mathcal{H} = \begin{bmatrix} d_5 & d_3 & d_1 & 0 & 0 & 0 \\ d_6 & d_4 & d_2 & d_0 & 0 & 0 \\ 0 & d_5 & d_3 & d_1 & 0 & 0 \\ 0 & d_6 & d_4 & d_2 & d_0 & 0 \\ 0 & 0 & d_5 & d_3 & d_1 & 0 \\ 0 & 0 & d_6 & d_4 & d_2 & d_0 \end{bmatrix}. \tag{5.28}$$

For $d_6 > 0$ all six leading principal determinants must be positive. The executable analysis reports every determinant and pole for each candidate. Speed limits, changing waypoints and camera-map updates remain nonlinear effects beyond this local calculation.

5.8 Discrete-time stability at the real control interval

Keep the physical state $x_p = [\phi, p, u_{\mathrm{act}}]^{\mathsf{T}}$ and integral state I. During one held command, obtain the exact linear plant update $x_{p,k+1} = A_d x_{p,k} + B_d u_k$ by solving the linear ODE over 2 ms. The software uses the current error in its candidate integral:

$$e_k = k_a(\phi_{c,k} - \phi_k) - p_k, \tag{5.29}$$

$$I_{k+1} = I_k + \Delta e_k, \tag{5.30}$$

$$u_k = K_p e_k + K_i I_{k+1} = (K_p + K_i\Delta)e_k + K_i I_k. \tag{5.31}$$

Let $C_e = [-k_a, -1, 0]$. At zero reference the combined update is therefore

$$\begin{bmatrix} x_{p,k+1} \\ I_{k+1} \end{bmatrix} = \begin{bmatrix} A_d + B_d(K_p + K_i\Delta)C_e & B_d K_i \\ \Delta C_e & 1 \end{bmatrix} \begin{bmatrix} x_{p,k} \\ I_k \end{bmatrix}. \tag{5.32}$$

This explicitly includes the integration ordering used in C. Strict sampled stability requires every eigenvalue z_i to satisfy $|z_i| < 1$. Yaw uses the same construction without the angle state; altitude uses $C_e = [-1, 0, 0]$ and velocity feedback $-K_d v$. Estimator dynamics, computation delay and allocation limits are additional effects not present in this particular test. The numerical analysis also repeats it with one control sample of input delay and reports the resulting spectral radius rather than asserting timing safety.

5.9 Include the 20 Hz companion with the 500 Hz flight loop

For one horizontal axis use physical state $x_p = [\theta_d, p_d, u_{\text{act}}, x, v]^{\mathsf{T}}$. Its hover plant is

$$\dot{x}_p = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & b_x & 0 & 0 \\ 0 & 0 & -1/\tau_m & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ g\pi/180 & 0 & 0 & 0 & -\gamma \end{bmatrix} x_p + \begin{bmatrix} 0 \\ 0 \\ 1/\tau_m \\ 0 \\ 0 \end{bmatrix} u. \tag{5.33}$$

The $g\pi/180$ entry explicitly converts the degree-valued angle into the radian-valued small-angle force law. Obtain A_d, B_d for one held 2 ms command, append the PI integral, and apply the same current-error integral ordering as above. With $k = K_p + K_i\Delta$, $C_e = [-k_a, -1, 0, 0, 0]$,

$$F = \begin{bmatrix} A_d + B_d k C_e & B_d K_i \\ \Delta C_e & 1 \end{bmatrix}, \qquad G = \begin{bmatrix} B_d k k_a \\ \Delta k_a \end{bmatrix}. \tag{5.34}$$

Thus $x_{k+1} = Fx_k + G\theta_c$ for the six-state augmented system. The companion holds $\theta_c = C_g x_k$ for $N = 25$ flight updates, where

$$C_g = -\frac{180}{\pi g}[0, 0, 0, k_x, k_v, 0]. \tag{5.35}$$

Substitute the recurrence a second time: $x_{k+2} = F^2 x_k + (F + I)G\theta_c$. A third substitution gives $F^3 x_k + (F^2 + F + I)G\theta_c$. Continuing the same physical recurrence for 25 held steps gives

$$x_{k+N} = F^N x_k + \left(\sum_{j=0}^{N-1} F^j G \right) C_g x_k = M x_k. \tag{5.36}$$

The numerical analysis checks every eigenvalue of M. With one complete guidance-period delay, append the old held command and check

$$M_{\text{delayed}} = \begin{bmatrix} F^N & \sum_{j=0}^{N-1} F^j G \\ C_g & 0 \end{bmatrix}. \tag{5.37}$$

This is an explicit 500 Hz/20 Hz sampled-system calculation. It still assumes exact state feedback, linear hover dynamics and scheduled updates. It is not a proof for the Kalman estimator, delayed perception, saturation, or Windows meeting every real-time deadline.

5.10 Frequency-domain margins and uncertainty

For the rate loop, open-loop $L_r = CG = b(K_p s + K_i)/[s^2(\tau_m s + 1)]$. At the gain crossover $|L(j\omega_c)| = 1$, phase margin is $180^\circ + \arg L(j\omega_c)$. At a -180° phase crossover, gain margin is $1/|L(j\omega_\pi)|$. These are frequency-domain local robustness measures, not obstacle-clearance margins. A pure delay adds phase $-\omega T_d$; the rough additional-delay allowance from a positive

phase margin is $T_d \simeq \mathrm{PM}_{\mathrm{rad}}/\omega_c$ for the crossing being considered. Multiple crossovers and unstable open-loop cases need care.

Plant uncertainty enters directly: $b_x \propto aT_{\max}/J_x$ and $b_h \propto T_{\max}/m$. The executable analysis varies these gains and τ_m over a declared finite grid and counts stable/unstable cases for the complete characteristic polynomials. A grid with no failures is evidence only at those points; it is not a continuous robust-stability proof. Battery sag, thrust asymmetry, flexible arms and estimator bias remain unmodelled physical risks until measured. Time-domain runs then test saturation, takeoff, landing and perception-driven commands beyond the linear analysis.

Chapter 6

Companion missions, boost and actuator authority

6.1 Keep one flight-control mode; move mission sequencing upstream

The flight core still receives roll/pitch angles, yaw rate and centered vertical-speed throttle. The companion selects mission phases: ground, spool-up in simulation, takeoff, hover, navigate, approach/hold, descent and landed. A mission phase does not introduce a new flight PID. The companion converts a requested height into bounded vertical speed; the existing altitude PID integrates that speed into its internal target. Start and finish points are explicit local coordinates. The start must agree with the measured launch position; selecting it must never teleport the aircraft state.

Ground contact supplies an upward normal reaction that balances weight before lift-off. At $T_z > mg$ that unilateral reaction vanishes and upward acceleration begins. Landing is the reverse transition with low descent speed and contact confirmation; simply forcing a trajectory through $h = 0$ is not a contact model. The 3D simulator handles contact; real landed detection remains a separate sensor/state-machine problem. A simulation-only motor gate cannot be presented as an implemented physical arming system.

6.2 What arming and RT boost mean

Arming is a deliberate permission to produce physical motor outputs. It is not the same as compiling a control function or connecting a battery. Physical arming stays disabled as requested. Takeoff/landing button gestures are mission requests; a future authorized safety supervisor must accept them before motors can be enabled. The simulated actuator gate is explicitly isolated from hardware.

RT is already present in the radio frame as a normalized trigger. Its proposed boost increases the maximum commanded vertical speed from 2 to 3 m/s and the nominal collective ceiling from 0.65 to 0.80 as RT moves from zero to one. The internal boost allowance rises over at least 0.25 s and decays over at least 0.10 s for a full-scale change. RT alone does not command a climb at centered vertical stick. The companion simulation inhibits boost during approach/landing. Roll/pitch leveling and yaw-rate control remain active. It is not acrobatic rate mode and not an unconditional full-throttle override. The boost ceiling has less correction reserve than normal mode; maximum climb requires measured thrust/energy reserve and later hardware tuning.

6.3 Prove the motor-headroom allocation interval

Let desired motor values before clipping be $u_i = c + d_i$, where d_i are the four differential roll/pitch/yaw corrections. For each motor to satisfy $0 \le u_i \le 1$, rearrange to $-d_i \le c \le 1 - d_i$. Intersect all four intervals:

$$-\min_i d_i \le c \le 1 - \max_i d_i. \tag{6.1}$$

This interval exists exactly when $\max d_i - \min d_i \le 1$. If the requested differential span exceeds one, uniformly scale all d_i by $\alpha = 1/(\max d_i - \min d_i)$; otherwise $\alpha = 1$. Clamp collective to the resulting feasible interval and calculate $u_i = c + \alpha d_i$. This preserves differential-command ratios, unlike clipping each motor independently. It cannot create missing thrust or guarantee full simultaneous torque authority. A future allocator should feed allocation residuals back into anti-windup; the current PID anti-windup covers each loop's own output limit, not the complete coupled allocation constraint.

Chapter 7

Exactly six feedback controllers

7.1 Signals and units

The six objects in the C file are roll angle, roll rate, pitch angle, pitch rate, yaw rate and altitude. A PID object may have a zero integral or derivative coefficient. Calling all six "PIDs" describes the general structure; the chosen outer loops are P and the chosen angular-rate loops are PI. No seventh vertical-rate PID is hidden in the altitude loop.

Angular gains below use **degrees** and degrees per second because that is the C interface. Physics uses radians. If an angular measurement is changed from degrees to radians, multiply corresponding numeric gains by $180/\pi$ to preserve the same normalized control output. Outer angle-to-rate P gains remain unchanged when both angle and rate units are converted together.

For a generic PID,

$$e_k = r_k - y_k, \tag{7.1}$$

$$I_k^{\text{cand}} = \text{clip}(I_{k-1} + e_k\Delta, -I_{\max}, I_{\max}), \tag{7.2}$$

$$u_k^{\text{raw}} = K_p e_k + K_i I_k^{\text{cand}} - K_d \dot{y}_k, \tag{7.3}$$

$$u_k = \text{clip}(u_k^{\text{raw}}, -u_{\max}, u_{\max}). \tag{7.4}$$

The derivative on measurement avoids setpoint derivative kick. All selected angular K_d values are zero. The altitude controller instead uses the available derivative reference v_c: $K_d(v_c - \hat{v}_z)$. The integrator candidate is accepted if unsaturated, or if the error drives an existing saturation back toward the admissible interval. Integrator and output limits are independent physical quantities.

7.2 Roll and pitch rate PI derivation

Near hover neglect cross coupling and lag for the initial design. With angular rate measured in deg/s,

$$\dot{p}_d = b_x u_\phi, \quad b_x = \frac{4aT_{\max}}{J_x}\frac{180}{\pi}, \qquad \dot{q}_d = b_y u_\theta, \quad b_y = \frac{4aT_{\max}}{J_y}\frac{180}{\pi}. \tag{7.5}$$

The plant transfer function from normalized torque command to rate is b/s. The PI controller is $C(s) = K_p + K_i/s = (K_p s + K_i)/s$. Unity feedback gives

$$1 + C(s)G(s) = 0, \tag{7.6}$$

$$1 + \frac{b(K_p s + K_i)}{s^2} = 0, \tag{7.7}$$

$$s^2 + bK_p s + bK_i = 0. \tag{7.8}$$

Choose a desired second-order polynomial $s^2 + 2\zeta\omega_n s + \omega_n^2$. Matching like powers of s proves

$$\boxed{K_p = \frac{2\zeta\omega_n}{b}, \qquad K_i = \frac{\omega_n^2}{b}, \qquad K_d = 0.} \tag{7.9}$$

For the baseline $\zeta = 0.9$, $\omega_n = 12$ rad/s. The roots have real part $-\zeta\omega_n < 0$. A state-space proof is also possible: with error $e = r - p_d$ and integral I, for constant r,

$$\frac{\mathrm{d}}{\mathrm{d}t}\begin{bmatrix} e \\ I \end{bmatrix} = \begin{bmatrix} -bK_p & -bK_i \\ 1 & 0 \end{bmatrix}\begin{bmatrix} e \\ I \end{bmatrix}. \tag{7.10}$$

Its determinant $\det(sI - A) = s^2 + bK_p s + bK_i$ gives the same condition.

Including the actuator lag changes the characteristic polynomial to

$$\tau_m s^3 + s^2 + bK_p s + bK_i. \tag{7.11}$$

For positive coefficients, the cubic Routh condition additionally requires $bK_p > \tau_m bK_i$, or $K_p > \tau_m K_i$. With the pole-matched gains this is $\omega_n < 2\zeta/\tau_m = 45$ rad/s. Both the baseline 12 and selected 15.6 rad/s designs meet this continuous linear condition. It is not a proof for saturation, delays, estimators or the full nonlinear aircraft.

7.3 Outer angle loops and yaw

The outer commands are

$$p_c = 3(\phi_c - \hat{\phi}), \qquad q_c = 3(\theta_c - \hat{\theta}) \quad [\text{deg/s}], \tag{7.12}$$

clamped to ± 180 deg/s. Under the ideal approximation $p_d = p_c$, $\dot{e}_\phi = -3e_\phi$, so $e_\phi(t) = e_\phi(0)e^{-3t}$. The time constant is $1/3$ s and the ideal two-percent settling time is $-\ln(0.02)/3 \simeq 1.304$ s. Real cascaded dynamics are slower/different. The inner angular bandwidth is intentionally higher than this outer loop.

Yaw has *only* rate feedback in the flight core:

$$\dot{r}_d = b_\psi u_\psi, \quad b_\psi = \frac{4\kappa T_{\max}}{J_z}\frac{180}{\pi}, \quad K_{p\psi} = \frac{2(0.9)\omega_\psi}{b_\psi}, \quad K_{i\psi} = \frac{\omega_\psi^2}{b_\psi}. \tag{7.13}$$

Baseline $\omega_\psi = 4$ rad/s. There is no yaw-angle PID in the flight controller. The companion optionally creates a bounded yaw-rate request from its local-heading error; that is a navigation policy, not an extra flight PID.

7.4 One altitude PID with vertical-speed stick command

For normalized centered throttle $s_t \in [0, 1]$,

$$v_c = 4(s_t - 0.5) \quad \text{m/s}, \qquad h_{c,k+1} = h_{c,k} + v_c\Delta. \tag{7.14}$$

At centre stick, $v_c = 0$ and the target height is held. The target is initialized to the estimated height and limited to within 2 m of it to avoid unlimited reference accumulation. There is one altitude control law:

$$e_h = h_c - \hat{h}, \quad \dot{I}_h = e_h, \tag{7.15}$$

$$\delta c = K_{ph}e_h + K_{ih}I_h + K_{dh}(v_c - \hat{v}_z), \tag{7.16}$$

$$c = \mathrm{clip}\left(\frac{c_0 + \delta c}{\max(0.5, \cos\hat{\phi}\cos\hat{\theta})}, 0, 1\right). \tag{7.17}$$

The tilt factor approximately preserves vertical thrust. The factor limit prevents numerical blow-up; it is not valid recovery control at large tilt. The nominal hover command is

$$c_0 = \frac{mg}{4T_{\max}} = 0.4903325. \tag{7.18}$$

The board scaffold still has a 0.50 placeholder; SITL explicitly supplies the model-based value. A real craft requires a measured hover thrust curve.

At fixed target, near hover, let $b_h = 4T_{\max}/m = 20$ m/s^2 per command. Then $\ddot{h} = b_h \delta c$. Substitution of PID feedback gives

$$s^3 + b_h K_{dh} s^2 + b_h K_{ph} s + b_h K_{ih} = 0. \tag{7.19}$$

Choose $(s + p_h)(s^2 + 2\zeta_h \omega_h s + \omega_h^2)$, multiply explicitly:

$$s^3 + 2\zeta_h \omega_h s^2 + \omega_h^2 s + p_h s^2 + 2p_h \zeta_h \omega_h s + p_h \omega_h^2 \tag{7.20}$$

$$= s^3 + (2\zeta_h \omega_h + p_h) s^2 + (\omega_h^2 + 2p_h \zeta_h \omega_h) s + p_h \omega_h^2. \tag{7.21}$$

Coefficient matching yields

$$\boxed{\begin{aligned} K_{dh} &= (2\zeta_h \omega_h + p_h)/b_h, \\ K_{ph} &= (\omega_h^2 + 2p_h \zeta_h \omega_h)/b_h, \\ K_{ih} &= p_h \omega_h^2 / b_h. \end{aligned}} \tag{7.22}$$

Baseline $\omega_h = 1.5$, $\zeta_h = 0.9$, $p_h = 0.8$ gives $(K_p, K_i, K_d) = (0.2205, 0.09, 0.175)$. Units are respectively command/m, command/(m s), and command s/m. The integral-state bound is 2 m s and the altitude-correction bound is 0.35 command. Actuator lag increases the order:

$$\tau_m s^4 + s^3 + b_h K_d s^2 + b_h K_p s + b_h K_i. \tag{7.23}$$

For positive coefficients the quartic Routh conditions reduce to

$$b_h K_d > \tau_m b_h K_p, \qquad (b_h K_d)(b_h K_p) > \tau_m (b_h K_p)^2 + b_h K_i. \tag{7.24}$$

These hold for the baseline and selected simulation gains. Discrete feedback and filtering still require the actual closed-loop tests.

7.5 A reproducible gain family, not hidden hand tuning

Three candidates use scale $\lambda \in \{0.7, 1, 1.3\}$ with $\omega_n = 12\lambda$, $\omega_\psi = 4\lambda$, $\omega_h = 1.5\lambda$, $p_h = 0.8\lambda$; outer P stays 3. Thus rate K_p scales as λ, rate K_i as λ^2, altitude K_p as λ^2, altitude K_i as λ^3, and altitude K_d as λ. Full numeric tables and the declared selection score appear in the results chapter. The selected family is the best of these three under that score, not a globally optimal tune.

Chapter 8

Kalman filtering: proof and implemented equations

8.1 Derivation of the gain

Let $\hat{x}^-$ be the prior estimate, $e^- = x - \hat{x}^-$, with covariance P^-. A measurement is $z = Hx + v$ with independent noise covariance R. The innovation is $\nu = z - H\hat{x}^- = He^- + v$. Hence

$$S = \mathrm{Cov}(\nu) = HP^-H^\mathsf{T} + R. \tag{8.1}$$

Use $\hat{x}^+ = \hat{x}^- + K\nu$. Then

$$e^+ = (I - KH)e^- - Kv. \tag{8.2}$$

Taking its covariance, retaining both independent-noise contributions, gives the Joseph form

$$P^+ = (I - KH)P^-(I - KH)^\mathsf{T} + KRK^\mathsf{T}. \tag{8.3}$$

Expanding this expression gives $P^+ = P^- - KHP^- - P^-H^\mathsf{T}K^\mathsf{T} + KSK^\mathsf{T}$. The expected squared error is $\mathbb{E}[e^{+\mathsf{T}}e^+] = \mathrm{tr}(P^+)$. Complete the square using $K_* = P^-H^\mathsf{T}S^{-1}$:

$$P^+ = P^- - P^-H^\mathsf{T}S^{-1}HP^- + (K - K_*)S(K - K_*)^\mathsf{T}. \tag{8.4}$$

The last term is positive semidefinite when S is positive definite. Its trace is minimized by $K = K_*$, proving the Kalman gain. Gaussianity is not required for this best linear update; Gaussian priors/noises additionally make it the conditional-mean Bayesian update and justify normal intervals.

8.2 Scalar one-angle filter

A strictly scalar attitude model (calibrated gyro bias assumed known) is

$$\hat{\theta}_k^- = \hat{\theta}_{k-1}^+ + \Delta(\omega_{g,k} - b), \tag{8.5}$$

$$P_k^- = P_{k-1}^+ + \Delta^2\sigma_g^2, \tag{8.6}$$

$$K_k = \frac{P_k^-}{P_k^- + R_\theta}, \tag{8.7}$$

$$\hat{\theta}_k^+ = \hat{\theta}_k^- + K_k(\theta_{a,k} - \hat{\theta}_k^-), \tag{8.8}$$

$$P_k^+ = (1 - K_k)^2P_k^- + K_k^2R_\theta. \tag{8.9}$$

This is the literal one-state interpretation of "1D Kalman". K is not an arbitrary fixed mixing percentage: it changes with prior uncertainty, sample interval and measurement noise.

8.3 Actual MPU filter: one axis, two states

The C implementation estimates *one angle at a time*, independently for roll and pitch, but also estimates gyro bias. Its state dimension is therefore two, even though it is often informally called a "1D attitude filter". This document names that distinction explicitly.

$$x = \begin{bmatrix} \theta \\ b \end{bmatrix}, \quad F = \begin{bmatrix} 1 & -\Delta \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} \Delta \\ 0 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & 0 \end{bmatrix}. \tag{8.10}$$

The bias is subtracted from the gyro, so the upper-right entry is negative. With independent per-sample gyro variance σ_g^2 and continuous bias random-walk spectral density q_b,

$$Q = \begin{bmatrix} \sigma_g^2\Delta^2 + q_b\Delta^3/3 & -q_b\Delta^2/2 \\ -q_b\Delta^2/2 & q_b\Delta \end{bmatrix}. \tag{8.11}$$

The bias contribution follows by integrating $q_b[-\tau\ 1]^\mathsf{T}[-\tau\ 1]$ from zero to Δ. Expanding FPF^T entry by entry gives

$$P_{00}^- = P_{00} - \Delta(P_{01} + P_{10}) + \Delta^2 P_{11} + \sigma_g^2\Delta^2 + q_b\Delta^3/3, \tag{8.12}$$

$$P_{01}^- = P_{01} - \Delta P_{11} - q_b\Delta^2/2, \tag{8.13}$$

$$P_{10}^- = P_{10} - \Delta P_{11} - q_b\Delta^2/2, \tag{8.14}$$

$$P_{11}^- = P_{11} + q_b\Delta. \tag{8.15}$$

Since $H = [1, 0]$, the matrix inverse in the gain formula is just a scalar division:

$$S = P_{00}^- + R_\theta, \quad K = \frac{1}{S}\begin{bmatrix} P_{00}^- \\ P_{10}^- \end{bmatrix}, \quad \begin{bmatrix} \hat\theta^+ \\ \hat b^+ \end{bmatrix} = \begin{bmatrix} \hat\theta^- \\ \hat b^- \end{bmatrix} + \begin{bmatrix} K_\theta \\ K_b \end{bmatrix}(\theta_a - \hat\theta^-). \tag{8.16}$$

K_θ is dimensionless; K_b has units s^{-1} and is normally negative with this sign convention. A positive angle innovation should reduce an estimated positive gyro bias that has been over-subtracted.

The initialization is $P_0 = \mathrm{diag}(1, 0.01)$, $\sigma_g^2 = 0.04$, $q_b = 0.0001$ and $R_\theta = 4$ in degree-based units. At $\Delta = 0.002$ s, the first prediction yields

$$P_{00}^- = 1.000000200000267, \tag{8.17}$$

$$P_{01}^- = P_{10}^- = -0.0000200002, \tag{8.18}$$

$$P_{11}^- = 0.0100002, \tag{8.19}$$

$$S = 5.000000200000267, \tag{8.20}$$

$$K_\theta \simeq 0.200000032, \qquad K_b \simeq -0.00000400004\ \mathrm{s}^{-1}. \tag{8.21}$$

The numerical first gain is not the eventual gain; plots and end-of-run values are generated from the actual C trace.

8.4 Accelerometer observations and their limitation

For calibrated specific force f_B in units of g, the small-acceleration gravity observations are

$$\phi_a = \mathrm{atan2}(f_y, f_z), \qquad \theta_a = \mathrm{atan2}(-f_x, \sqrt{f_y^2 + f_z^2}). \tag{8.22}$$

Convert these to degrees before feeding the C filter. The accelerometer measures specific force, not gravity alone: $f_B = R^\mathsf{T}(a_W + ge_3)$ in SI units. During sustained translation, these angle observations are biased. Fixed $R_\theta = 4\ \mathrm{deg}^2$ is a provisional robustness choice, not the variance implied directly by 0.005 g sensor noise. Their nonlinear Jacobian would propagate raw measurement covariance; motion adds further model error. This explains why a mathematically valid covariance recursion alone does not prove that the reported confidence intervals are calibrated in real flight.

8.5 Altitude filter: state, measurements and expanded products

Use $x = [h, v_z]^\mathsf{T}$ and the exact F, G derived earlier. Corrected world-up acceleration is the control input. The C state prediction is

$$\hat{h}^- = \hat{h} + \Delta\hat{v}_z + \tfrac{1}{2}\Delta^2 a_z, \qquad \hat{v}_z^- = \hat{v}_z + \Delta a_z. \tag{8.23}$$

The covariance equations from $FPF^\mathsf{T} + Q_{\text{held}}$ are

$$P_{00}^- = P_{00} + \Delta(P_{01} + P_{10}) + \Delta^2 P_{11} + \tfrac{1}{4}\sigma_a^2\Delta^4, \tag{8.24}$$
$$P_{01}^- = P_{01} + \Delta P_{11} + \tfrac{1}{2}\sigma_a^2\Delta^3, \tag{8.25}$$
$$P_{10}^- = P_{10} + \Delta P_{11} + \tfrac{1}{2}\sigma_a^2\Delta^3, \tag{8.26}$$
$$P_{11}^- = P_{11} + \sigma_a^2\Delta^2. \tag{8.27}$$

Barometric height has $z_h = Hx + v_h$, $H = [1, 0]$, $R_h = 0.0025$ m^2 in SITL. The gains are

$$K_h = \frac{P_{00}^-}{P_{00}^- + R_h}, \qquad K_v = \frac{P_{10}^-}{P_{00}^- + R_h}. \tag{8.28}$$

Here K_v is positive: a higher observed altitude tends to raise both estimated height and upward speed. A barometer reading is corrected *once when fresh*, every ten control steps. Reusing the same reading as if it were ten independent measurements would understate uncertainty. The gains logged between observations are the most recent gains, not ten new corrections.

SITL starts at $P_0 = \text{diag}(0.04, 0.1)$, with $\sigma_a = 1.5$. The first 2 ms prediction gives

$$P^- = \begin{bmatrix} 0.040000400009 & 0.000200009 \\ 0.000200009 & 0.100009 \end{bmatrix}, \tag{8.29}$$
$$S = 0.042500400009, \tag{8.30}$$
$$K_h \simeq 0.941177024, \qquad K_v \simeq 0.00470605 \text{ s}^{-1}. \tag{8.31}$$

The firmware scaffold's boot defaults are instead $P_0 = \text{diag}(1, 1)$ and $R_h = 2$ m^2. The host simulation deliberately supplies the documented test configuration through its reset interface. Neither is a measured BMP280 noise model. They must not be confused when reproducing results.

8.6 Joseph multiplication in full detail

Let a symmetric prior be $P^- = \begin{bmatrix} a & b \\ b & d \end{bmatrix}$, measurement variance r, and gain $K = [k_0, k_1]^\mathsf{T}$. For $H = [1, 0]$,

$$M = I - KH = \begin{bmatrix} 1 - k_0 & 0 \\ -k_1 & 1 \end{bmatrix}. \tag{8.32}$$

Writing $z = 1 - k_0$ and multiplying row by column,

$$MP^- = \begin{bmatrix} za & zb \\ b - k_1 a & d - k_1 b \end{bmatrix}, \tag{8.33}$$
$$MP^-M^\mathsf{T} = \begin{bmatrix} z^2 a & z(b - k_1 a) \\ z(b - k_1 a) & d - 2k_1 b + k_1^2 a \end{bmatrix}, \tag{8.34}$$
$$KrK^\mathsf{T} = r \begin{bmatrix} k_0^2 & k_0 k_1 \\ k_0 k_1 & k_1^2 \end{bmatrix}. \tag{8.35}$$

Adding gives exactly the four expressions implemented in `covariance_correct`. The routine averages the two off-diagonal prior entries to remove small numerical asymmetry, and writes the same posterior off-diagonal value twice. Joseph form is a sum of positive semidefinite matrices in exact arithmetic. Finite-precision roundoff still requires monitoring; a failed covariance validity check must not be disguised by claiming the physical filter is healthy.

8.7 Gravity compensation, lidar and observability

The upward acceleration supplied to the filter is

$$a_z = -\sin\hat{\theta}\, f_x + \sin\hat{\phi}\cos\hat{\theta}\, f_y + \cos\hat{\phi}\cos\hat{\theta}\, f_z - g, \tag{8.36}$$

with f in m/s^2. At rest and level it is zero. In the CPU model this uses the preceding C attitude estimate, not the exact plant attitude, so attitude error affects height estimation. The resulting correlated errors are not fully represented by this two-state covariance model.

For flat ground and a truly downward aligned rangefinder, approximate $h = d\cos\phi\cos\theta$ after accounting for the sensor lever arm. Its variance can be approximated by $R_h = J_d \Sigma J_d^{\mathsf{T}}$, with

$$J_d = \begin{bmatrix} \cos\phi\cos\theta & -d\sin\phi\cos\theta & -d\cos\phi\sin\theta \end{bmatrix}. \tag{8.37}$$

Terrain slope, missing returns, tilt and range limits violate this simple measurement model. A downward TOFSense-F2 Mini is therefore a useful future height aid, but it is not fused in the current trace. A front sensor sees only one ray; it cannot establish a full safe corridor by itself.

For the altitude pair, the two-observation observability matrix is

$$\mathcal{O} = \begin{bmatrix} H \\ HF \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & \Delta \end{bmatrix}, \quad \det\mathcal{O} = \Delta \neq 0. \tag{8.38}$$

Thus repeated independent height measurements make both height and speed observable. The analogous angle-bias determinant is $-\Delta$. With all height measurements missing, integration alone drifts and no longer bounds height uncertainty. Loss of sensor aiding requires a defined, tested degraded mode; generic zero motor output in flight is not a safe landing policy.

Chapter 9

Uncertainty, statistical checks and error accounting

9.1 What standard deviation does and does not mean

For a scalar zero-mean Gaussian estimation error $e \sim \mathcal{N}(0, P)$, $\sigma = \sqrt{P}$. The standardized variable $u = e/\sigma$ has unit normal density $\exp(-u^2/2)/\sqrt{2\pi}$. Therefore

$$\Pr(|e| \le c\sigma) = \int_{-c}^{c} \frac{e^{-u^2/2}}{\sqrt{2\pi}} \mathrm{d}u, \tag{9.1}$$

which is approximately 68.27% for $c = 1$ and 95.45% for $c = 2$. A covariance is a model prediction, not a measured error bound. Gaussian intervals can fail under bias, bad calibration, outliers, latency or incorrect process-noise assumptions.

For N Monte Carlo errors e_i at a fixed time, compute

$$\bar{e} = \frac{1}{N}\sum_i e_i, \quad s_e = \sqrt{\frac{\sum_i (e_i - \bar{e})^2}{N-1}}, \quad \mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_i e_i^2}. \tag{9.2}$$

The $N - 1$ denominator makes sample variance unbiased for independent samples. RMSE includes bias; standard deviation describes spread about the sample mean. For $N = 500$ and true coverage 0.9545, the binomial standard error of an observed two-sigma coverage fraction is $\sqrt{0.9545(1 - 0.9545)/500} \simeq 0.00932$ at one independent time point. Successive time points within a trajectory are correlated, so pooling them does not create that many independent trials.

9.2 Innovation consistency and physical-parameter errors

For a scalar observation, normalized innovation squared is NIS $= \nu^2/S$. Under the correctly specified linear Gaussian model, its mean is one and its distribution is χ^2_1. Persistent large values suggest underestimated uncertainty, bias, bad timing or sensor failures; very small values can indicate overestimated noise. This is a diagnostic, not an automatic proof of sensor validity. The C trace records angle innovation and its variance. Altitude innovation variance can be reconstructed on fresh corrections from $S = R_h/(1 - K_h)$, subject to roundoff and $K_h < 1$.

For a physical parameter vector η with covariance Σ_η, a first-order output uncertainty is

$$\Sigma_f \simeq J_f \Sigma_\eta J_f^{\mathsf{T}}, \qquad (J_f)_{ij} = \frac{\partial f_i}{\partial \eta_j}. \tag{9.3}$$

For example, $c_0 = mg/(4T_{\max})$ has derivatives

$$\frac{\partial c_0}{\partial m} = \frac{g}{4T_{\max}}, \qquad \frac{\partial c_0}{\partial T_{\max}} = -\frac{mg}{4T_{\max}^2}. \tag{9.4}$$

With independent relative errors this gives

$$\left(\frac{\sigma_{c_0}}{c_0}\right)^2 \simeq \left(\frac{\sigma_m}{m}\right)^2 + \left(\frac{\sigma_T}{T_{\max}}\right)^2. \tag{9.5}$$

For $b_x \propto aT_{\max}/J_x$, the logarithmic first-order perturbation is $\delta b_x/b_x \simeq \delta a/a + \delta T/T - \delta J_x/J_x$. Correlations must be included if, for example, moving a battery changes both centre of mass and inertia. The current mass/thrust perturbations are scenario tests, not measured probability distributions for the actual aircraft.

9.3 Separate error sources

The report distinguishes: floating-point and timestep error; noisy sensors; calibration error; structural model error; uncertain mass/inertia/thrust; latency and stale-data error; and perception/localization error. Running the same approximate equations very precisely only reduces numerical error. It does not eliminate structural or identification error. Six successful scenarios provide evidence for those scenarios, not a probability of safe flight in arbitrary environments.

Chapter 10

Companion navigation and perception

10.1 A bounded static-map baseline

The common C++ planner discretizes $x \in [-2, 10]$, $y \in [-4, 4]$ into 0.2 m cells. A circle obstacle of radius r_i blocks centres within $r_i + 0.65$ m. The 0.65 m inflation includes the 0.34 m propeller envelope and additional tracking/grid allowance. Eight-connected A* uses edge length 0.2 or $0.2\sqrt{2}$ and Euclidean heuristic. Diagonal motion through blocked corners is rejected. Since Euclidean straight-line distance never exceeds path length, the heuristic is admissible. It does not prove continuous-space safety under arbitrary tracking errors; actual trajectory clearance is computed separately.

For waypoint error e_{xy}, set desired horizontal velocity $v_c = \mathrm{clip}_{\|\cdot\| \le 0.8}(e_{xy})$ and acceleration $a_{xy,c} = \mathrm{clip}_{\mathrm{component},1.4}[1.8(v_c - v_{xy})]$. With yaw ψ, transform world horizontal acceleration into body heading:

$$\begin{bmatrix} a_{xB} \\ a_{yB} \end{bmatrix} = \begin{bmatrix} \cos\psi & \sin\psi \\ -\sin\psi & \cos\psi \end{bmatrix} \begin{bmatrix} a_{xW} \\ a_{yW} \end{bmatrix}. \tag{10.1}$$

The approximate requests are $\theta_c = \arctan(a_{xB}/g)$ and $\phi_c = -\arctan(a_{yB}/g)$, limited to 15 degrees. The waypoint advances within 0.4 m. Goal acceptance requires horizontal error below 0.2 m, vertical error below 0.2 m and horizontal speed below 0.15 m/s, followed by one second dwell in the route test. Flights begin already airborne at 1.5 m; these tests do not include takeoff, landing or recovery from a crash.

10.2 ROS 2 data contract

The ROS C++ source uses a custom timestamped AutonomySetpoint message with roll angle, pitch angle, yaw rate, centered throttle, validity and goal status. Angles are radians at the ROS boundary and degrees inside the C core. A 250 ms odometry freshness gate invalidates companion output. Local pose and velocity must use one consistent frame; the wrapper requires a documented adapter that transforms standard body-frame Odometry twist to world velocity. Ordinary body twist must not be silently interpreted as world velocity. No setpoint arms hardware. A future physical bridge must enforce pilot mode authority, timestamp/sequence checks, watchdogs and a tested degraded mode.

The executed DDS test uses two Python adapter nodes with numeric test envelopes, carrying sensor samples, commands, motor proxies and estimator diagnostics. It exercises actual DDS serialization/transport and C execution, but does not validate generated custom-message C++ bindings, physical serial framing or a complete real-time companion system.

10.3 Why YOLO is not a range sensor

For a calibrated pinhole camera, optical coordinates satisfy $u = f_x X/Z + c_x$ and $v = f_y Y/Z + c_y$. If a registered depth image supplies Z,

$$X = (u - c_x)Z/f_x, \qquad Y = (v - c_y)Z/f_y. \tag{10.2}$$

Without depth or another geometric constraint, multiplying (X, Y, Z) by any positive scale leaves (u, v) unchanged. This proves the metric-scale ambiguity of a single detection image. A bounding box cannot supply reliable distance by itself. The tested geometry utility rejects invalid range values and back-projects *all* valid sampled depth pixels, including objects outside YOLO's recognized classes. Bounding-box median depth is only a semantic annotation.

Transform each point with calibrated extrinsics and synchronized pose: $p_W = R_{WB}(R_{BC}p_C + t_{BC}) + t_{WB}$. Future map insertion needs ground filtering, voxelization, obstacle growth, clearing/decay, temporal consistency and replanning. None of those steps can be replaced by the standalone bus-image detection test. The same ROS C++ navigation core can run on Pi or Jetson; perception acceleration and camera drivers remain platform-specific concerns.

Chapter 11

Reproduction and next physical measurements

The evidence is dated 20 October 2026. Run the host C checks, build SITL libraries, execute the route suite, execute the ROS DDS adapter test, and render graphs/video using the scripts listed in the results chapter. All stochastic seeds and assumed parameters are saved alongside the traces. The video plays one simulated second per second; producing it offline is not proof of live 500 Hz execution. DDS wall-clock percentiles include Windows scheduling.

To make the model physically precise, measure complete takeoff mass and centre of mass; identify motor/propeller thrust and current versus command at several battery voltages; estimate actuator lag; measure inertia or assemble component inertias; measure vibration, gyro Allan behaviour, accelerometer and barometer noise after mounting; measure communication age/jitter; calibrate camera intrinsics/extrinsics and rangefinder offsets. Until then all numeric PID values in this report are simulation candidates. The LiPo's capacity and C-rating alone do not establish usable energy, voltage sag, thrust reserve or safe cell condition.

Primary implementation references for later integration are NVIDIA's Isaac Sim requirements and Ultralytics' prediction and model export documentation. The algebra and gain derivations above are developed explicitly for this project.

Chapter 12

Recorded numerical results

12.1 Six PID gain tables

These values are read from the saved simulation summary. Order is roll angle, roll rate, pitch angle, pitch rate, yaw rate, altitude. All angular interfaces here are degree-based. The original baseline used $\lambda = 1$ and its selected airborne video loaded $\lambda = 1.3$ through the host API. The new conservative default is $\lambda = 0.7$, reported in the later mission chapter. These historical tables are retained for comparison; none is a measured physical flight tune.

12.1.1 Baseline

Controller	K_p	K_i	K_d
Roll angle	3	0	0
Roll rate	0.00118476874	0.00789845828	0
Pitch angle	3	0	0
Pitch rate	0.00118476874	0.00789845828	0
Yaw rate	0.00767944893	0.0170654412	0
Altitude	0.220500007	0.0900000036	0.174999997

12.1.2 Selected simulation

Controller	K_p	K_i	K_d
Roll angle	3	0	0
Roll rate	0.00154019939	0.013348395	0
Pitch angle	3	0	0
Pitch rate	0.00154019939	0.013348395	0
Yaw rate	0.00998328347	0.0288405959	0
Altitude	0.372644991	0.197730005	0.227500007

12.2 Selection and route suite

Every candidate is run on the same nominal seed 2350. A successful candidate must avoid collision, end within 0.3 m of the 3D goal, and satisfy the navigator's arrival conditions followed by one second dwell. Among successful nominal candidates, the declared score is

$$J = t_f + 20\,\mathrm{RMSE}_h + 10\,\|p_f - p_g\| + \frac{2}{d_{\min} + 0.01}. \tag{12.1}$$

The coefficients are preference weights with implicit unit conversion; this is not a dimensionless universal cost or a statistical estimator. Ranking depends on clearance definition and weights. After including the complete propeller envelope, the selected nominal case is nominal-1.3.

Case	t_f [s]	Height RMS [m]	Final error [m]	Clearance [m]	Success
nominal-0.7	21.45	0.01057	0.13837	0.29061	yes
nominal-1.0	21.50	0.01115	0.14035	0.29897	yes
nominal-1.3	21.50	0.01199	0.14165	0.30058	yes
heldout-heavy	21.55	0.02968	0.14497	0.31306	yes
heldout-weak-thrust	21.60	0.03310	0.14106	0.29882	yes
heldout-light	20.35	0.03887	0.03652	0.27065	yes

Held-out scenarios do not change the selected gains: heavy mass is $1.15m$ (seed 2351), weak thrust is $0.85T_{\max}$ (seed 2352), and light mass/strong thrust is $0.85m$, $1.1T_{\max}$ (seed 2353). The controller hover feed-forward is kept nominal, so the altitude integrator must compensate. These three perturbations are not an exhaustive robustness region. Wind is a one-second horizontal force of $(0.4, -0.3, 0)$ N between 7 and 8 seconds. All saved scenarios have zero motor-command saturation fraction. Initial motor thrust is balanced for the physical test mass because these are airborne route tests, not spool-up tests.

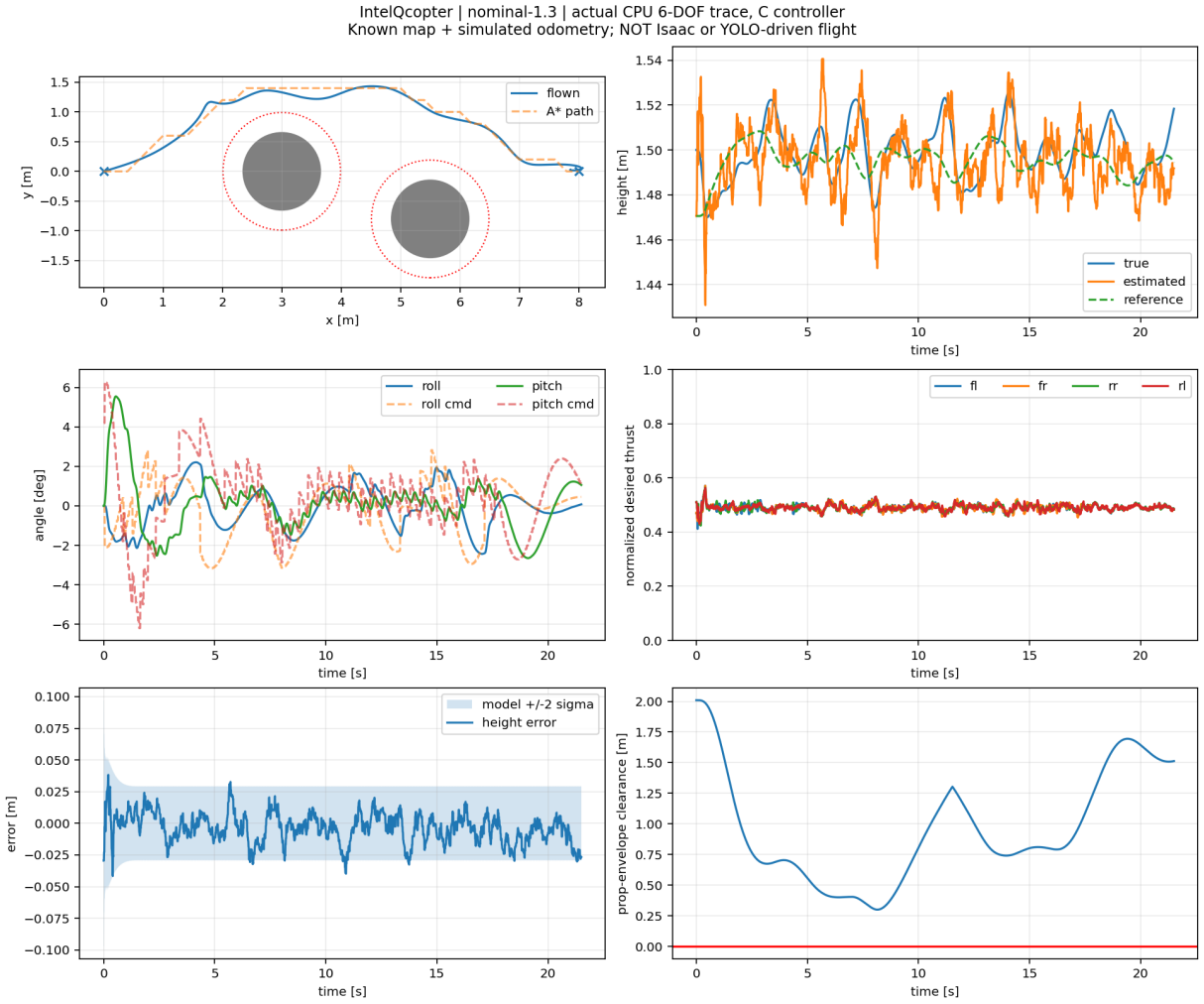


Figure 12.1: Original plots of saved CPU trace. Dashed circles include the propeller envelope; the A* planner uses a larger planning inflation.

12.3 Kalman gains, standard deviation and distributions

The final logged gains in the selected route are

$$K_\theta = 0.00096639531, \qquad K_b = -0.000223559371 \text{ s}^{-1}, \tag{12.2}$$

$$K_h = 0.0834309161, \qquad K_v = 0.181649998 \text{ s}^{-1}. \tag{12.3}$$

These are end-of-run time-varying values, not preselected fixed constants. The selected route's roll and pitch estimation RMSEs are 0.75897 and 1.29857 degrees. Height error lies within its predicted two-sigma interval at 98.419% of logged times. This is one correlated nonlinear trajectory and must not be interpreted as 500 independent statistical trials.

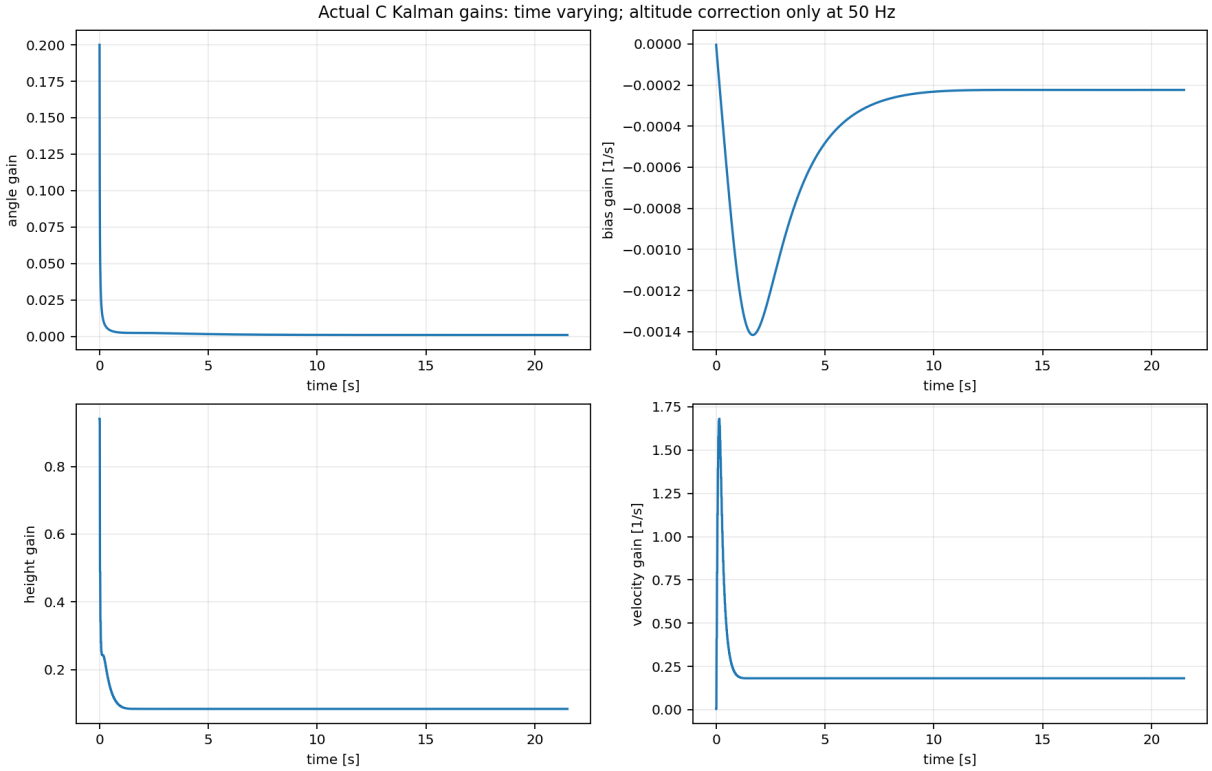


Figure 12.2: Actual gain evolution. Altitude gains are held between fresh barometer observations.

12.3.1 Independent Monte Carlo experiments

Both experiments call the production C estimator functions, with 500 synthetic independent trials of 2500 steps (5 seconds each). The angle test uses seed 2350, prior angle sigma 1 degree, bias sigma 0.1 deg/s, gyro sample sigma 0.2 deg/s, accelerometer-angle sigma 2 degrees, and bias random-walk sigma 0.01 deg/s/$\sqrt{\text{s}}$. It includes a prescribed eight-degree ramp. The altitude test uses seed 2352, height prior sigma 0.2 m, speed prior sigma $\sqrt{0.1}$ m/s, unobserved independent acceleration errors of sigma 1.5 m/s^2 per sample, and height observations of sigma 0.05 m at 50 Hz. The altitude truth is propagated by exactly the held-acceleration stochastic model used to derive Q. This tests statistical consistency under a matched model, not realistic unknown airframe physics.

At final logged time	Mean error	Empirical std	Predicted std	2σ coverage
Angle [deg]	-0.004526	0.078910	0.079017	95.00%
Height [m]	-0.000296	0.013305	0.014442	96.80%

The angle generator samples bias Brownian motion at each endpoint while the covariance includes the continuous interval integral; the difference is small at 2 ms, but the test is not an exact continuous-noise sampler. Normalized innovation checks and real sensor data are still needed before using these intervals as engineering confidence claims.

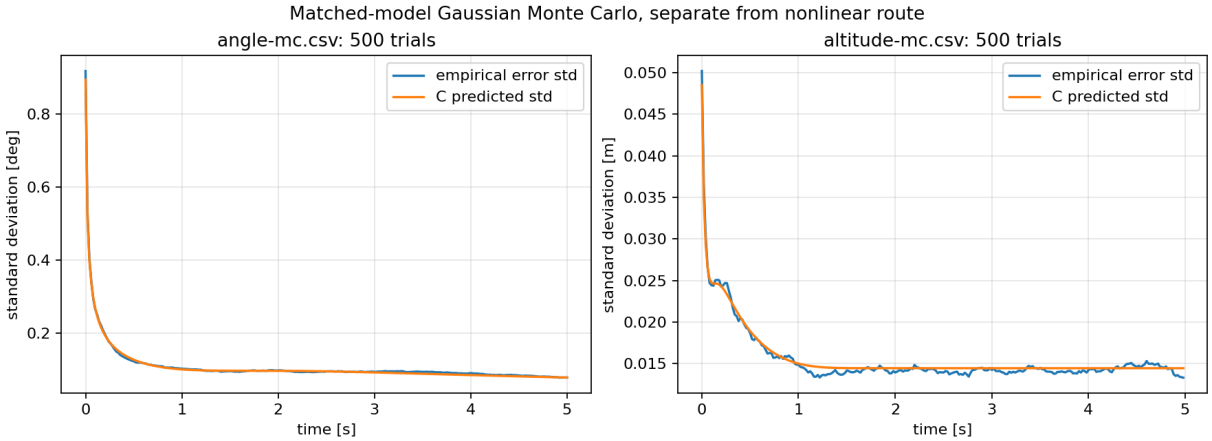


Figure 12.3: Empirical spread versus model-predicted spread; original C Monte Carlo data.

12.4 Numerical refinement and algebra checks

Host tests independently expand FPF^{T} and the Joseph correction, check covariance symmetry and nonnegative determinants, and check positive pitch increases rear thrust in FLU coordinates. Separate tests check rotation orthogonality/determinant, matrix noncommutativity, nilpotence, stale navigation, NaN navigation, out-of-map goals, occupied goals, blocked maps, depth-plane back-projection and rejection of invalid depth. Linearized angular-rate and altitude polynomials including the motor lag have strictly left-half-plane roots for both baseline and selected gains. This does not include outer-loop, estimator or sampled-data stability in that root test.

The selected case is repeated with unchanged 500 Hz sensing and control but physical integration at 1 ms and 0.5 ms. The maximum aligned-time position differences from the original 2 ms integration are:

Physics interval	Max difference [m]	Route success
1.0 ms	0.0017292	yes
0.5 ms	0.0025954	yes

The comparison bounds disagreement between these tested discretizations only. It is not a rigorous global integration-error bound and does not measure physical model error. Noise samples remain identical at the control interval.

12.5 Executed ROS 2 communication test

Two same-process Python adapter nodes exchanged 10,751 strictly sequenced request/response frames through Fast DDS UDPv4. Every received sensor/command envelope invoked the production C core; the result returned through DDS to the CPU plant. The best gain scale was used and the route succeeded. The median wall-clock round trip was 0.4693 ms, the 99th percentile 0.7514 ms, and the maximum 14.0538 ms. The maximum can exceed the 2 ms control period; this is not a hard-real-time 500 Hz scheduling demonstration. The RP2350 is intended to own that local loop; slow companion work must not block it. The two-node test is not an independent-process or wireless benchmark and does not include a real UART. The complete C++ ROS wrappers have been authored but not built on this PC.

12.6 Executed YOLO and USD tests

YOLO11n performed actual GPU inference on the official bus reference image: 5 detections. The warm measured inference phase reported 9.850 ms and the end-to-end warm call took 24.618 ms on this PC. One reference image is neither a perception benchmark nor a navigation safety test. Model and input SHA-256 hashes are saved in the JSON report. The separately executed OpenCV DNN CPU test loaded the exported ONNX model, applied the C++ node's square-letterbox/raw-output/NMS contract, and produced 5 detections in an inference call of 97.894 ms. This exercised the portable inference engine through Python, not a running ROS C++ node. The raw output shape was $1 \times 84 \times 8400$ and the model hash is saved. The ROS C++ YOLO source supports a raw YOLO11 ONNX model, letterboxing, confidence thresholding, NMS and synchronized registered-depth annotation. That C++ wrapper has not been compiled or executed here. The separate portable USD scene is authored with positive assumed mass/inertia, four 9047 rotor visuals, a conservative collision envelope, ground and obstacles. Its parser and attributes are validated separately; it is not an executed Isaac scene.

12.7 Files and reproducibility

Project paths below are relative to the repository root, not shell commands that install system software.

C firmware `flight_controller/flight_controller.c`.

Host checks `tools/run_flight_controller_tests.ps1`.

Host libraries `tools/build_sitl.ps1`.

CPU route `sim/sitl/run.py`.

Algebra/refinement `sim/sitl/analysis_tests.py`.

DDS integration `sim/ros2_integration_test.py` with bundled Isaac Python 3.12.

Perception reference `sim/yolo_reference_test.py`.

USD authoring `sim/isaac/build_stage.py`.

Graphs/video `sim/sitl/visualize.py`.

Data `artifacts/simulation/2026-10-20/summary.json`, per-case CSVs, Monte Carlo CSVs, `analysis-tests.json`, DDS and perception subdirectories.

Original logs `artifacts/test-runs/2026-10-20/`.

Video `artifacts/simulation/2026-10-20/best-flight-cpu.mp4`.

The video visualizes the selected saved trajectory at real-time playback speed with live-progress graphs. It is not an Isaac recording, physical flight, or a demonstration of YOLO-driven obstacle avoidance. Isaac remains blocked by the recorded GPU startup failure on this workstation.

Chapter 13

New ground-to-ground garden experiments and numerical analysis

13.1 What changed from the original report

The general matrix-multiplication and matrix-exponential lessons have been removed. Their instructional style is applied to the actual free-body equations, linearization, transfer functions, full feedback polynomials and stability proofs. The earlier known-map airborne test is retained as historical evidence. The new experiment starts on the ground, takes off, navigates, approaches, descends and lands with actual contact dynamics. Flight control still has exactly six PID objects and the same angle/rate/vertical-speed interface.

The current boot gain candidate is $\lambda = 0.7$, replacing the earlier $\lambda = 1$ default. This choice follows both the wider local stability checks and a completed mission. It is a provisional simulation/bench candidate; it is not a hardware flight tune. Host tests explicitly load their own gains when needed.

13.2 Current six-controller candidate and model coefficients

Controller	K_p	K_i	K_d
Roll angle	3	0	0
Roll rate	0.000829338154	0.0038702446	0
Pitch angle	3	0	0
Pitch rate	0.000829338154	0.0038702446	0
Yaw rate	0.00537561392	0.008362066	0
Altitude	0.108044997	0.0308699999	0.122500002

Angular units are degrees and degrees/s, as in the C code. Normalized output is a desired-thrust proxy. The assumed gains from allocation and mechanics are $b_x = b_y = 18231.40540218$ deg/s^2/command, $b_\psi = 937.56730112$ deg/s^2/command, and $b_h = 20$ m/s^2/command. The actuator lag is 0.04 s and translational drag rate is 0.1 s^{-1}. For the conservative candidate, the following polynomials are formed from the equations in the derivation chapters. They retain actuator lag; height additionally retains drag. Write $T = N/D$ with descending powers:

$$N_\phi(s) = 45.36s + 211.68, \tag{13.1}$$

$$D_\phi(s) = 0.04s^4 + 1s^3 + 15.12s^2 + 115.92s + 211.68. \tag{13.2}$$

$$N_r(s) = 5.04s + 7.84, \tag{13.3}$$

$$D_r(s) = 0.04s^3 + 1s^2 + 5.04s + 7.84. \tag{13.4}$$

$$N_h(s) = 2.45s^2 + 2.1609s + 0.6174, \tag{13.5}$$

$$D_h(s) = 0.04s^4 + 1.004s^3 + 2.55s^2 + 2.1609s + 0.6174. \tag{13.6}$$

13.3 Poles, step response, Routh values and sampling

The table reports actual local transfer-function responses, including numerator zeros. Settling means the last exit from a two-percent band over the simulated 20 s linear response; it is not a nonlinear aircraft landing time.

Scale	Loop	Overshoot [%]	Settling [s]	Sampled radius
0.7	angle	0.000	1.385	0.9949320
0.7	roll_rate	35.902	0.426	0.9863007
0.7	yaw	19.007	1.763	0.9938399
0.7	height	20.572	3.692	0.9986314
1	angle	0.000	1.344	0.9944676
1	roll_rate	46.751	0.553	0.9846446
1	yaw	21.687	1.195	0.9908625
1	height	22.207	4.027	0.9979595
1.3	angle	0.000	1.325	0.9942743
1.3	roll_rate	55.928	0.589	0.9867462
1.3	yaw	25.234	0.883	0.9900456
1.3	height	23.664	3.241	0.9973339

The local continuous poles for the conservative loops are listed explicitly:

angle $-5.89357 + 12.66510j,\ -5.89357 - 12.66510j,\ -10.67161 + 0.00000j,\ -2.54124 + 0.00000j$ s^{-1}.

roll_rate $-9.00000 + 13.07670j,\ -9.00000 - 13.07670j,\ -7.00000 + 0.00000j\ \mathrm{s}^{-1}$.

yaw $-18.87451 + 0.00000j,\ -3.06274 + 1.00198j,\ -3.06274 - 1.00198j\ \mathrm{s}^{-1}$.

height $-22.35501 + 0.00000j,\ -1.37553 + 0.00000j,\ -0.68473 + 0.18192j,\ -0.68473 - 0.18192j$ s^{-1}.

The saved JSON contains full numerator/denominator arrays, every Routh first column, phase margins for the rate loops, and sampled eigenvalues with and without one flight-sample delay. The sixth-order horizontal models have positive leading Hurwitz determinants for the tested candidates. Their multirate radii are:

Scale	Guidance	500 Hz/20 Hz radius	With 50 ms delay
0.7	navigate	0.9644378	0.9696527
0.7	approach	0.9522570	0.9552026
1	navigate	0.9654857	0.9708107
1	approach	0.9521239	0.9567542
1.3	navigate	0.9658711	0.9712471
1.3	approach	0.9520753	0.9573129

These tests omit estimator states and map latency. They must not be confused with proving real processor deadlines or global nonlinear stability.

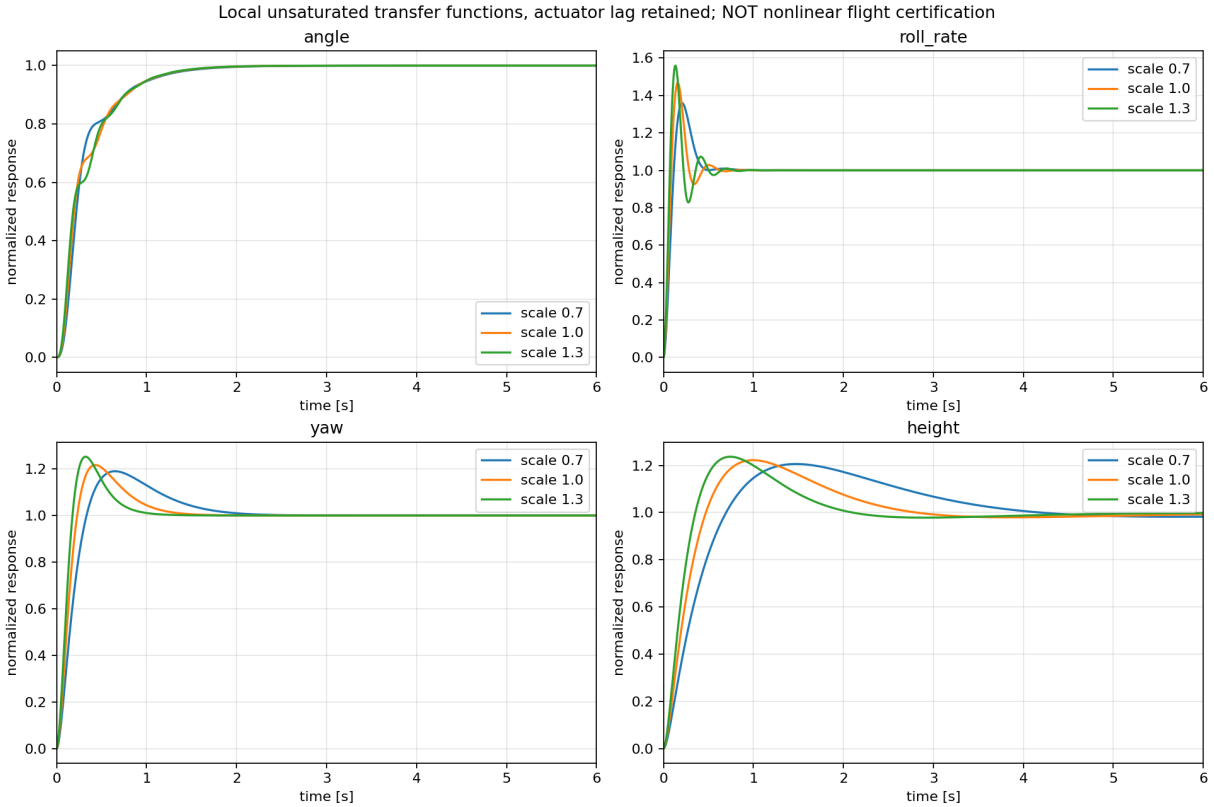


Figure 13.1: Local transfer-function step responses. Overshoot differs from the nominal second-order design because the complete models contain extra poles and zeros.

13.4 The uncertainty grid exposes a real tuning limitation

The finite grid uses fifteen plant-gain factors from 0.65 to 1.35 and fifteen actuator lags from 0.02 to 0.09s, for angle, yaw-rate and height models:

Scale	Stable tests	Total	Largest real pole
0.7	675	675	-0.473717
1.0	673	675	0.069313
1.3	634	675	0.686074

Thus the faster candidate is not robust over all these assumed points. The separate analytic interval argument in the stability chapter proves the local decoupled conservative-candidate inequalities throughout the stated continuous box, including a height-drag interval. The complete camera-guided nonlinear aircraft is not covered by that theorem. Actual motor lag remains unmeasured.

13.5 Camera-only panel distance, derived from the measurement geometry

The hardware direction is one RGB camera plus one downward 1D lidar. No depth image is supplied to this planner. For this controlled course only, fixtures are rectangular blue boards 1.2m wide and 1.8m high with white borders. Their visible blue inner rectangle is 1.09m by 1.69m. These are chosen test-fixture dimensions, not an assertion of a universal standard panel size.

YOLO identifies candidate image regions. Colour/contour fitting supplies four ordered inner corners; clipped or nonconvex quadrilaterals are rejected. Given calibrated intrinsics K, define planar object points

$$P_1 = (-w/2, -H/2, 0),\ P_2 = (w/2, -H/2, 0),\ P_3 = (w/2, H/2, 0),\ P_4 = (-w/2, H/2, 0). \tag{13.7}$$

Each corner satisfies

$$\lambda_i \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = K(R_{CO}P_i + t_{CO}). \tag{13.8}$$

The planar pose solver estimates R_{CO}, t_{CO}. Positive depth and a reprojection RMS below 2 pixels are required. A low reprojection error alone does not prove correct metric scale or remove planar pose ambiguity.

In the frontal special case, apparent height h_p pixels gives $h_p = f_y H/Z$, so $Z = f_y H/h_p$. Differentiate this physical equation:

$$\frac{\partial Z}{\partial f_y} = \frac{Z}{f_y}, \quad \frac{\partial Z}{\partial H} = \frac{Z}{H}, \quad \frac{\partial Z}{\partial h_p} = -\frac{Z}{h_p}. \tag{13.9}$$

For independent small calibration/size/pixel errors,

$$\frac{\sigma_Z^2}{Z^2} \simeq \frac{\sigma_f^2}{f_y^2} + \frac{\sigma_H^2}{H^2} + \frac{\sigma_p^2}{h_p^2}. \tag{13.10}$$

This proves why distant small images have larger relative range uncertainty. It also proves the unresolved scale problem: replacing both H and Z by 1.2 times their values leaves h_p unchanged. A regression test explicitly confirms that a 20% wrong fixture size produces a 20% wrong range scale. The four rasterized geometry tests at 2.5–5 m have errors saved in the report; they are synthetic calibration tests, not real camera accuracy measurements.

For the general four-corner fit, local pose covariance can be approximated by $(J^{\mathsf{T}}\Sigma_{\text{pix}}^{-1}J)^{-1}$ when the reprojection Jacobian has sufficient rank and one pose solution dominates. A multimodal/ambiguous solution must not be summarized as a single confident Gaussian. The implementation uses a provisional conservative range scale $\sigma_r = 0.04Z + 0.03\text{m}$ and obstacle inflation; this heuristic is not a calibrated posterior covariance.

Optical right/down/forward transforms to body forward/left/up as

$$(X_B, Y_B, Z_B) = (Z_C, -X_C, -Y_C). \tag{13.11}$$

Add the 0.18 m forward / 0.02 m upward camera lever arm, then use $p_W = R_{WB}(p_B + t_{BC}) + p_{WB}$. World-map uncertainty also includes pose and extrinsic calibration errors; simulated odometry supplies millimetre-scale translation noise and the true orientation in this prototype. A real VIO implementation is still missing.

13.6 Training and operating-domain limitations

The actual YOLO11n derivative was trained for 12 epochs at 320 pixels on 180 synthetic training images and 40 synthetic validation images. Validation poses are held out; the simulation tests also change panel layouts. This is not a real-world generalization test. Raw training output and its CSV are retained, including an initial plotting-dependency error; the missing dependency was repaired later. Real images, hard negatives, non-blue panels, lighting changes and unknown-size objects remain unvalidated. No true obstacle coordinates, camera depth or segmentation labels enter the online mapper. Renderer IDs are used only to generate training labels, and true geometry is used only for offline scoring.

The downward ray lidar is a simulated range sensor, not a validated TOFSense driver. On flat ground it supplies $h = (d + 0.085)\cos\phi\cos\theta$ with a known lever arm; invalid/near-ground returns are rejected. The C two-state height filter corrects fresh lidar observations with variance 0.0004 m^2 in addition to the barometer. Real surface reflectivity, slope, sunlight and sensor latency have not been identified. Virtual ground contact is used by the mission's touchdown confirmation; a reliable real landed detector remains unfinished.

13.7 Recorded mission progression and outcomes

The first attempt reached the ground without collision but missed the horizontal finish by about 0.48 m and failed the 0.4 m criterion. Position feedback during approach/landing fixed that problem. Later refinements require a stable approach before descent, inhibit RT boost during approach/landing, and correct the visible inner-panel dimensions. Earlier iteration files are preserved; not every early iteration has a source hash. The delivery run records source hashes at startup.

Run	Time[s]	Final XY[m]	Clearance[m]	PnP obs	Result
initial-landing-drift	35.35	0.47998	0.6998	37	fail
garden-nominal	35.35	0.05938	0.6998	37	pass
garden-heldout	32.05	0.03132	0.5597	44	pass
garden-camera-dropout	32.75	0.02545	0.7019	37	pass
garden-boost-slew	53.10	0.01703	0.4800	49	pass
garden-conservative	35.40	0.01896	0.5705	42	pass
garden-selected-ros2	32.20	0.00792	0.3612	39	pass
garden-final	35.45	0.00814	0.4904	37	pass

Different layouts, seeds and software iterations are not a controlled speed ranking. The delivered video selects the current conservative-gain, calibrated, shared-C++ policy run with actual ROS 2 selection/transport, rather than claiming a global optimum from those different courses. Boost can worsen mission settling; it is an allowance, not evidence of better-than-FPV performance. The delivery run takes 35.45s, ends 0.008137m from the requested XY finish, reaches 1.255538m and has minimum evaluated panel clearance 0.490441m including the conservative rotor-envelope collision shape. Height RMS error during hover/navigation/approach is 0.017921m, and height-estimation RMS error across the complete run is 0.003722m. These are conditional simulation values.

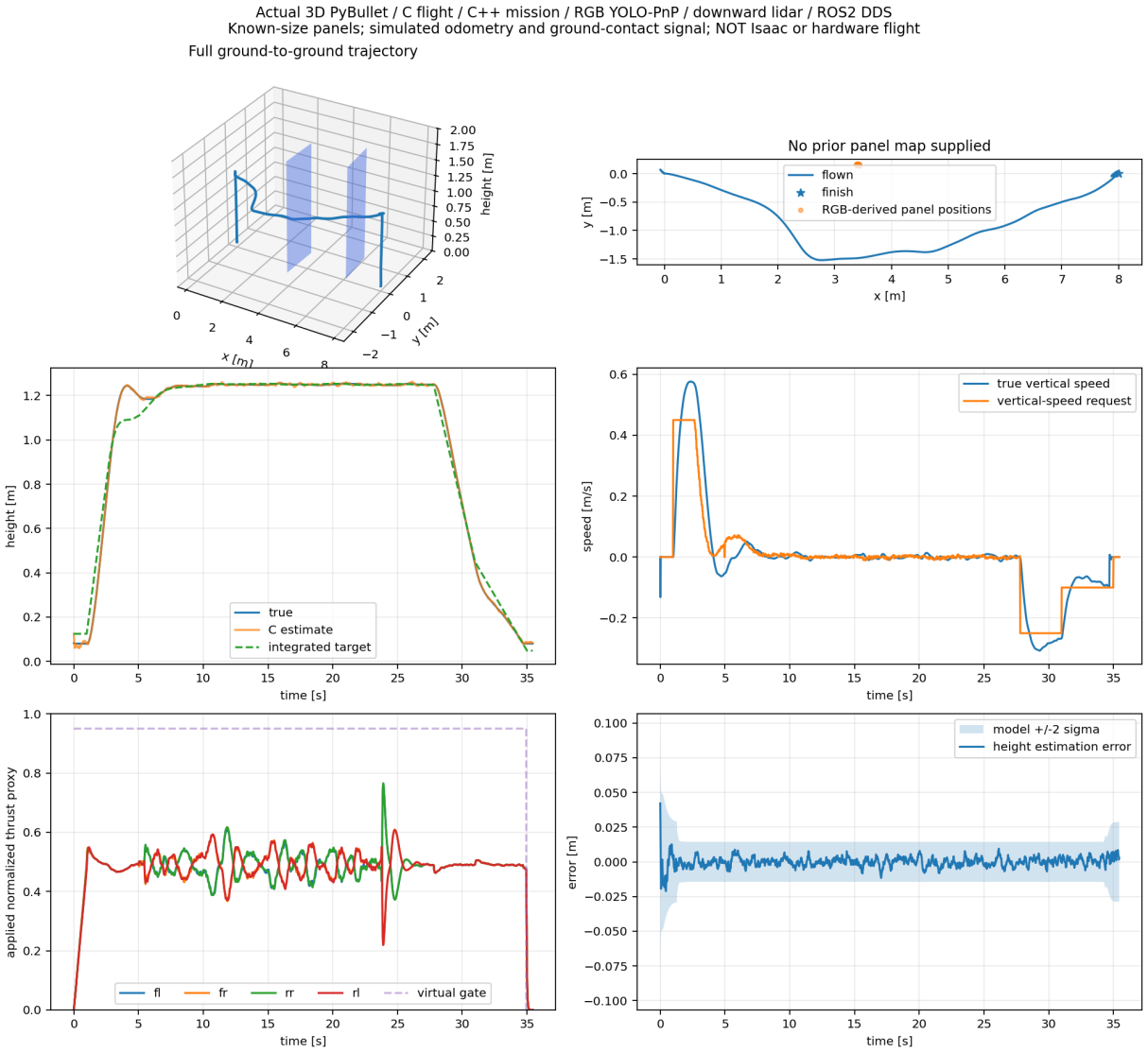


Figure 13.2: Original complete-flight data, including takeoff, landing and controller uncertainty. True geometry shown in evaluation figures is not supplied to the online planner.

13.8 ROS 2 selection, timing and implementation boundaries

The final adapter run completes 19,378 sequenced requests through two ROS 2 DDS nodes. Median measured request latency is 0.5113 ms, p99 is 1.5267ms and maximum is 22.9170 ms. The test includes C resets,500 Hz control steps and lidar corrections. A Python3.10 renderer reaches bundled Python3.12 ROS bindings through local TCP IPC; DDS then carries the test request to the C subscriber and returns its result. This is actual DDS execution, not proof that every Windows update meets 2 ms. The complete ROS C++ wrappers are authored but still unbuilt on this PC.

The working bundled selector accepts seven values: start XYZ, finish XYZ and cruise height in local metres. ROS acknowledges the chosen values, and the mission rejects a start that differs from the actual launch pose. The final demonstration uses a separate initial-condition spawn, not an in-flight teleport. The C++ package exposes RequestMission and a CLI client, plus a land-at-current-XY request service. Camera PnP publishes metric observations into the online map. No service, gesture or simulator gate enables physical motor arming.

13.9 Current missing information and unresolved problems

The separate project-status document orders the hardware questions. The main remaining requirements are: measured takeoff mass/centre of mass/inertia; motor/9047 thrust/current/lag data; exact ESC protocol; camera model and calibrated intrinsics/extrinsics/timing; an implemented real local-pose/VIO source; exact LoRa module/band and airtime budget; sensor drivers; routed PCB/power/current measurement; real landed detection and tested sensor-loss/battery authority. The garden is an original illustrative campus scene based on visual references, not a surveyed USTHB map. Single RGB plus one lidar ray does not establish general metric obstacle coverage without additional geometric assumptions or validated motion-based reconstruction. Physical motors remain disabled.